Curso BackEnd - 225h - Técnico em Desenvolvimento de Sistemas - SENAI

Profº Diogo TB

Escola SENAI Americana

2º Semestre 2026

Objetivos do Curso

- Desenvolver Aplicações web Server Side, utilizando a linguagem PHP;
- Aplicar Sisntaxe Nativa PHP (Vanilla);
- Manipulação HTTP;
- Persistência de Dados;
- Segurança contra SQL Injection/CSRF;
- Refatoração em POO (Programação Orientada ao Objeto);
- Arquitetura MVC (Model, View, Controller);
- Utilização do FrameWork Laravel;

obs: framework - um conjunto de bibliotecas que oferecem uma solução completa para o desenvolvimento de alguma coisa

Cronograma do Semestre

Carga Horária: 105h 1º Semestre e 120h 2º Semestre

Duração: 20 Semanas 1º Semestre e 20 Semanas 2º Semestre

Semana 1: Introdução ao BackEnd e Configuração do Ambiente PHP

O que é BackEnd?

O back-end é a parte de uma aplicação que o usuário não vê, mas que faz tudo funcionar por trás das telas.

O Back-End é a parte de um sistema que funciona nos servidores, sendo responsável por executar a lógica da aplicação, processar informações e armazenar dados.

Além disso, o BackEnd é responsável por atender ás solicitações do Frontend.

Sobre o mercado atual: o cenário é bom, mas mais exigente do que era. Quem conhece só o básico enfrenta mais concorrência. Quem alia backend sólido com IA aplicada, cloud e inglês está num patamar completamente diferente — vagas internacionais remotas são uma realidade pra esse perfil.

O Backend é formado pelo servidor, banco de dados, lógica de programação com APIs e linguagens de programação/frameworks. Esses componentes trabalham juntos para processar dados, armazenar informações e garantir o funcionamento da aplicação.

Para que serve

- Processar lógica de negócio: regras, cálculos, validações (ex: calcular frete, aplicar desconto, validar login)
- Gerenciar banco de dados: salvar, buscar, atualizar e deletar informações
- Autenticação e autorização: controlar quem pode acessar o quê (login, senhas, permissões)
- Fornecer APIs: criar "pontes" (endpoints) para o frontend ou outros sistemas consumirem dados
- Integração com serviços externos: pagamentos, e-mails, notificações, APIs de terceiros
- Segurança: proteger dados sensíveis, evitar ataques (SQL injection, XSS, etc.)

- Escalabilidade e performance: garantir que o sistema aguente muitos usuários ao mesmo tempo.

Principais Tecnologias Linguagens de programação: Ferramentas usadas para escrever o código do servidor, como Python, Node.js (JavaScript), Java e PHP.APIs: Os "caminhos" que permitem que o que você vê no celular converse com o servidor.

Areas de Atuação

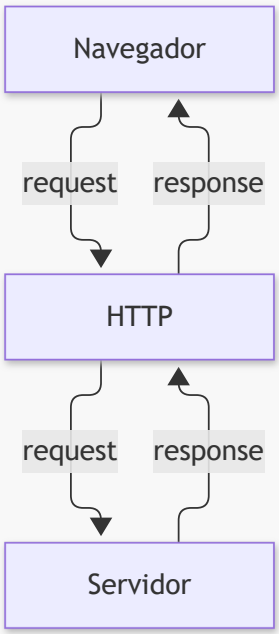
- Fintechs e Bancos
- Segurança, transações, alta escala
- E-commerce
- Catálogo, pedidos, pagamentos
- Healthtechs
- Prontuários, telemedicina
- SaaS / Startups
- Backend é o coração do produto
- Logística
- Rastreio, rotas, tempo real
- Educação
- Plataformas, conteúdo, usuários

O Ciclo de Vida da Requisição HTTP

O que é HTTP?

HTTP, Hypertext Transfer Protocol, é um protocolo de comunicação utilizado para transferência de informações na WWW (World wide Web) e em outros sistemas de redes.

O HTTP é a base para que o cliente e um servidor web troquem informações. Ele permite a requisição e a resposta de recurso como, imagens, arquivos e textos.



Como Funciona na Prática o BackEnd

- **Ação do Usuário**: Envia uma Solicitação pela UI(Interface do Usuário). Exemplo de UI: Tela do Celular, Navegador da Internet, Alexa, IOT ...
- **Enviar uma Requisição**: A UI transforma ação do Usuário em uma Requisição HTTP.

- **O Processamento BackEnd**: O Código BackEnd recebe o pedido, valida os dados e decide o que fazer. Ex: consultar uma informação no BD(Base de Dados).
- **Resposta**: O servidor devolde o resultado para a UI. Ex: Um Login Autorizado, Confirmação de uma Compra...

Tipos de Requisição HTTP

Os tipos de requisição HTTP indicam a ação que o usuário deseja executar no servidor. As principais ações são:

- **GET**: Pede dados de um lugar especifico do servidor. "Não Faz Alterações no Servidor"
- **DELETE**: Apaga um Dados do Servidor.
- **POST**: Envia dados novo para *criar* algo ou processar informações no servidor.
- **PUT/PATCH**: Modificaar um dados já existente.

Iniciando o PHP

PHP (HyperText PreProcessor) é uma linguagem de programação interpretada e open source, focada no desenvolvimento de sistemas para web, pode ser usada junto com HTML para criação de páginas web dinâmicas.

O PHP de fato é yma das linguagens de programação mais populares da atualidade. Ela permite que você crie aplicações web robustas, de uma muito simplificada e direta. A linguagem tem diversos recursos que facilitam e aceleram o ´processo de desenvolvimento de sites e sistemas para web. E além od mais, ela ainda tem um ótimo ecossistema, uma excelente comunidade e um grande mercado de trabalho.

Instalando o PHP

- Fazer o Download do PHP (php.net)
- ZIP - NTS(Non Thread Safe) 8.5
- Descompactar o Arquivo do PHP na pasta C:\src\php (Para Descompactar usar o 7Zip = Melhor) => nunca salvar arquivo ou programas na raiz do sistema(C:)
- Adicionar a Pasta do PHP(C:\src\php) as Variáveis de Ambiente do Sistema (PATH)
- Verificar a Instalação rodando o comando *php --version*

Criando Minha Primeira Aplicação em PHP

1. Antes de começar da Codar:

- Preparar meu VSCODE
    - Criar um Profile próprio para PHP
    - Instalar Extensões Necessária para Transformar o VSCode em uma IDE:
        - PHP Intelephense => permite a utilização de Snippets(atalhos de Código)
        - PHP Debug => ajuda a encontrar erros de código
        - PHP Cs Fixer => formatação de códigos (Identação)
        - PHP Server => ajuda na criação de um servidor local para PHP
    - Desabilitamos o PHP Nativo do VsCode (@builtin PHP)

2. Hello World (muito importante)

Semana 2 - Variáveis, Constantes e Operadores em PHP

Estudo de Variáveis e Constantes em PHP

Declarar variávies é alocar um espaço na memória que permite a inclusão e manipulação de dados.

Variávies

- devem ser declaradas usando "$" antes do nome da variável

- são não tipadas (não precisa declarar o tipo dela na criação) ,
- podem ser String, Numéricas (interger e float), Booleanas e Nulas. Não Permite declaração de Undefined
- Usar o "declare(Strict_types=1);" na primeira linha do arquivo; => blinda o sistema contra conflitos de tipos de variáveis

Constantes

- não poodem ser mudadas ou redeclaras após a criação
- pode ser criada usando "const" ou "define"
- não permite interpolação

Estudo de Operadores

Aritméticos: São usados para realizar Cálculso

Operador	Nome	Exemplo	Resultado
+	Adição	10+5	15
-	Subtração	10-5	5
*	Multiplicação	10*5	50
/	Divisão	10/5	2
%	Modulo(Resto)	10%3	1 (10 div 3 da 3, sobra 1)
**	Expoente	2**3	8(2 elevado a 3)

obs: O Operador % é o melhor amigo de um programador , permite ordenar listas e organizar fila e pilhas

Relacionais: Permite o Relacionamento entre dois ou mais valores, o resultado de uma operação é sempre uma booleana (verdadeiro ou falso).

Operador	Significado	Exemplo	Resultado
>	Maior que	18 > 18	false
>=	Maior ou igual a	18 >= 18	true
<	Menor que	10 < 20	true
<=	Menor ou igual a	10 <=5	false
==	Comparação de Valor	"10"==10	true
===	Comparação Estrita	"10"===10	false
!=	Diferente	"10"!=10	false
!==	Estritamente Diferente	"10"!==10	true

Lógicos: Permite a Combinação entre sentenças.

- Operador AND (E) => && : para o resultado ser verdadeiro, Todas as Combinações precisam ser verdadeira
    - true && true => true
    - true && false => false
- Operador OR (OU) => || : para o resultado ser verdadeiro, Basta apenas uma condição ser verdadeira
    - false || true => true
    - false || false => false

- Operador NOT (Não) => ! : Inverte a lógica da Operação,
    - !true => false
    - !false => true

Semana 3 - Estrutura de Controle de Dados (Condicionais e Repetição)

- **Conteúdo**: Estrutura `if`, `else`, `elseif`, operadores ternários, `match` => substituto do `switch/case`, loops `for`, `while`, `do-while` e `foreach`

Estruturas de Controle da Dados Ajudam no Processo de Automatização em Programas e Sistemas

Condicionais (IF, ELSE, ELSEIF)

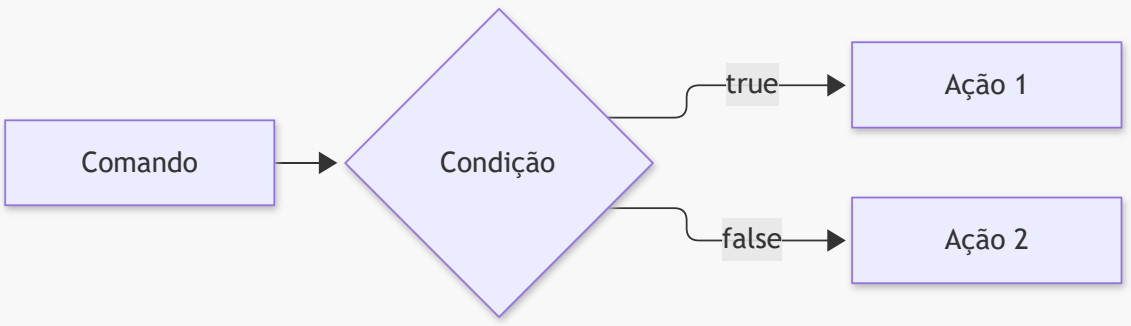
Formas de Uso

- uso do `if` apenas: Exemplo: aplicar desconto de 10% em compras acima de 100 Reais;




```php
if($valorCompra > 100){
    $valorFinal = $valorCompra * 0.9;
}
```

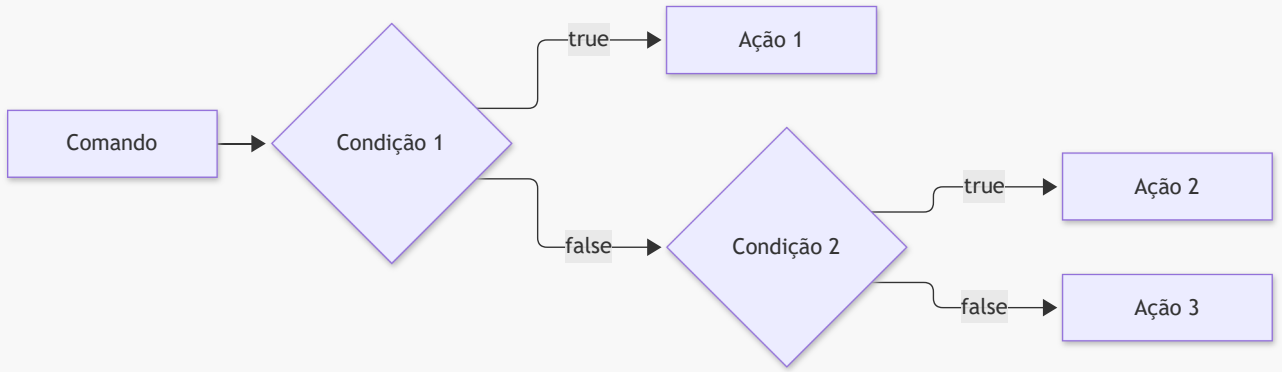
- Uso do `if`e do `else` Exemplo: Aplicar um desconto de 10% para compras acima de 100reais e 5% para as demais compras




```php
if($valorCompra > 100){
```

```
    $valorFinal = $valorCompra * 0.9;
} else {
    $valorFinal = $valorCompra * 0.95;
}
```

- Uso `elseif` (If Encadeado) => Estrutura usada para manipulação de dados em duas ou mais condicionais. Exemplo: Compras acima de 200 reais tem 15% de desconto, compras acima de 10 reais tem 10% de desconto e demais compras tem 5% desconto




```
if($valorCompra > 200) {
    $valorFinal = $valorCompra * 0.85;
} elseif($valorCompra > 100) {
    $valorFinal = $valorCompra * 0.9;
} else {
    $valorFinal = $valorCompra * 0.95;
}
```

obs: sempre usar `elseif` para situações que precisam de mais de uma condição, ou seja, fazer encadeamento das condições

- Uso *ERRADO* do if

```
if($valorCompra > 200) {
    $valorFinal = $valorCompra * 0.85;
}
if($valorCompra > 100) {
    $valorFinal = $valorCompra * 0.9;
} else {
    $valorFinal = $valorCompra * 0.95;
}
```

Operadores Ternários

Um atalho para a estrutura condicional `if/else, normalmente escrito em uma única linha de código.

`condição ? verdadeira : falsa`

Perfeito para decisões curtas de uma linha de comando

Exemplo: Verificar se a pessoa é maior de idade (18);

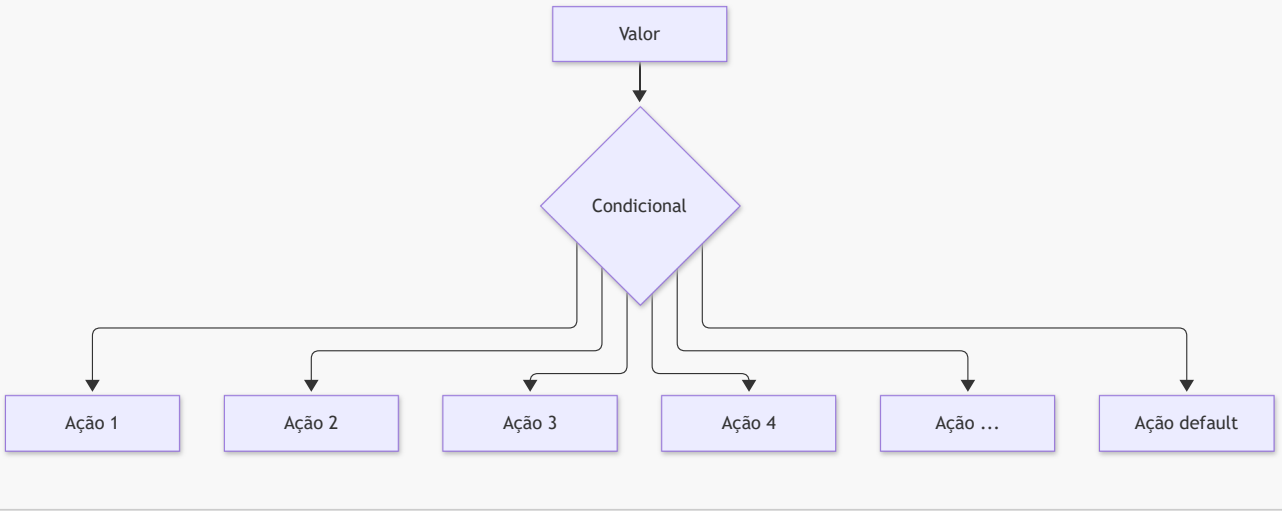
```php
$idade = 10;
//O formato é (Condição) ? Verdadeiro : Falso;

$status = ($idade>=18) ? "Maior de Idade" : "Menor de Idade";
$status2 = ($idade>=60) ? "Idoso" : ($idade>=18) ? "Adulto" : "Criança" ;

echo $status //
```

Expressão Condicional `match` (PHP 8)

No mercado atual de PHP, não se uma mais uma `Switch/Case` para chegar valores fixos, usa-se o `match`. Ele compara um valor e retoran diretamente o resultado caso atenda a condição.



Exemplo: Selecionar o Dia da Semana a partir de um Nº

```php
$diaSemanaNum = date("W"); // pega o Dia da Semana em formato numérico

$nomeDiaSemana = match($diaSemanaNu) {
    "0" => "Domingo",
    "1" => "Segunda",
    "2" => "Terça",
    "3" => "Quarta",
    "4" => "Quinta",
    "5" => "Sexta",
    "6" => "Sábado",
    "default" => "Dia Inválido"
};

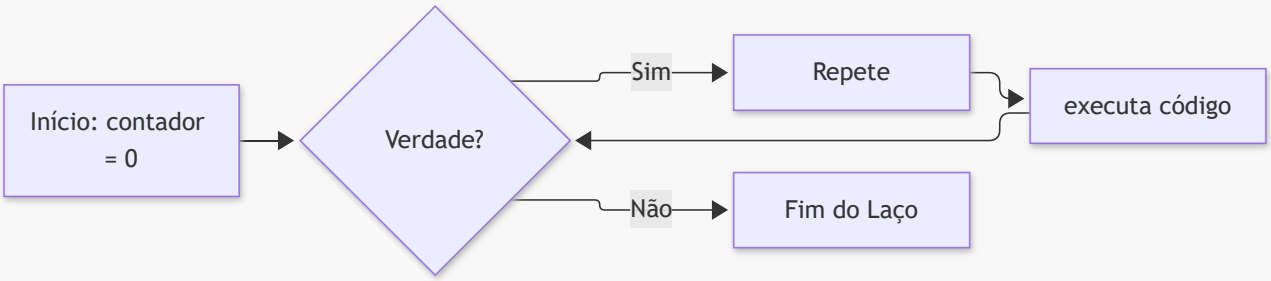
echo " Hoje é : $nomeDiaSemana";
```

Laços de Repetição

Um laço de repetição faz com que um bloco de código rode várias vezes até que uma condição mande parar.

- O Laço `while` (Enquanto)

Ele verifica se a condição é verdadeira ANTES de entrar no laço. Ideal quando você não sabe exatamente quantas vezes vai rodar o laço.



Exemplo de Aplicação do While: jogo de Adivinhação de um nº Secreto

```php
$numeroSecreto = rand(1,10);

$tentativas = 0;

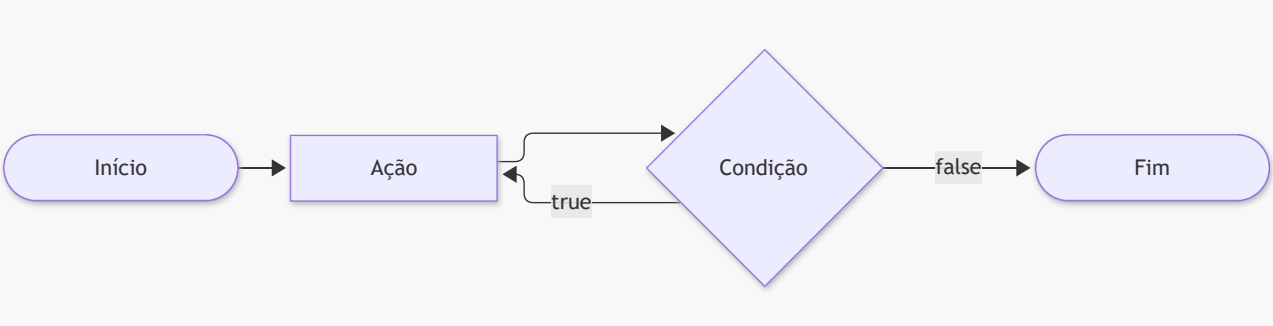
$numeroEscolhido = 0;

while(numeroEscolhido != numeroSecreto){
    echo "Tente Novamente"
    //vou Escolher outro Nº para adivinhar
    numeroEscolhido = rand(1,10);
    tentativas++;
}

echo "Acertou Miseravi!!! o nº secreto é $numeroEscolhido";
```

- O Laço `do-while` (Faça - Enquanto)

A diferença é que ele executa o bloco pelo menos uma vez, mesmo que a conduição seja false desde o início, pois ele só pergunta no final.



Exemplo: Jogo de Adivinhação de um nº

```php
$numeroSecreto = rand(1,10);

do{
    $numeroEscolhido = rand(1,10);

    if(numeroEscolhido == numeroSecreto){
        echo "Parabéns, Acertou!!!";
        break;
    }
    echo "Tente Novamente!!!";

} while(numeroEscolhido != numeroSecreto);
```

O Freio de Emergência: `break` e `continue`

As vezes precisamoso interferir no laço enquanto ele está rodando

- `break`=> **Para Tudo!** Quebra o laço interiro e avai embora
- `continue` => **Pula a rodada!** Ele ignora o código daquela rodada especifica e pula logo par a próxima repetição.

Exemplo de Aplicação do Código: Sistema de Controle do Elevador

```php
for($andar = 1 ; $andar<=10; $andar++){
    if($andar ==4){
        echo "Andar $andar está em obras. Passando direto!";
        continue;
    }

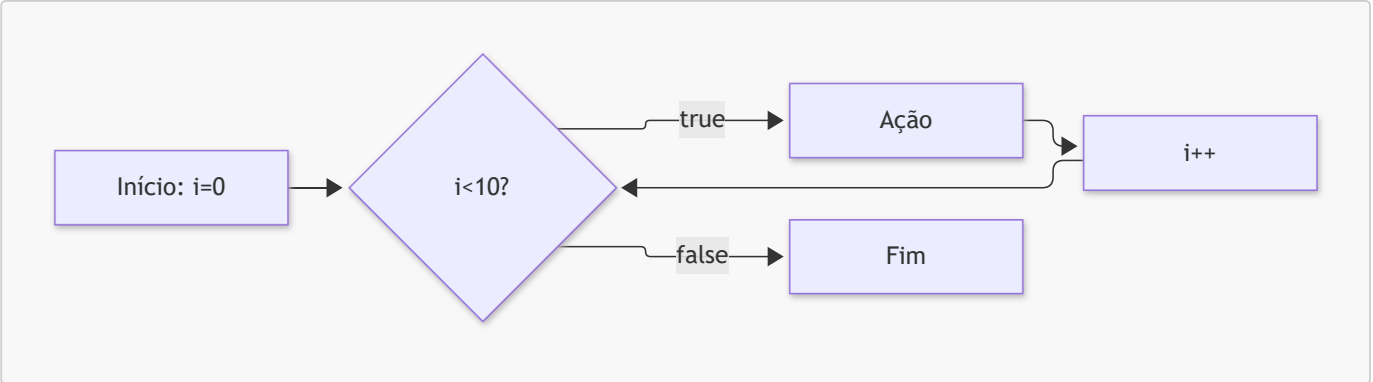
    echo "Elevador parou no andar $andar"
}
```

Laço de Repetição `for`

Use o `for`quando você sabe quantas vezes precisa repetir uma ação ou quando precisa controle um contador. Ele possui três partes:

- inicialização,
- condição,
- incremento;

for(inicialização; condição; incremento){ Ação }



Exemplo: Exibir todos os meses do Ano

```
for($mes=1; $mes<=12; $mes++){
    echo "Mês $mes";
}
```

Nesse Exemplo, `$mes`começa em 1, o laço continua enquantio `$mes`for menor ou igual a 12 e, ao final de cada repetição, `$mes++`aumenta o contador em 1.

Laço de Repetição `foreach`

Use o `foreach` quando precisar percorrer cada item de um *_array_*. Ele acessa os elementos diretamente, sem que você precise controlar o contador.

Exemplo: Imprimir todos os items de um vetor

```
$frutas = ["Maça", "Banana", "Uva", "Pera"];

foreach($frutas as $fruta){
    echo "Fruta: $fruta";
}
```

Outro Exemplo: Acessar a chave e o valor de cada item:

```
$precos = [
    "Caderno" => 25.90,
    "Caneta" => 5.50,
    "Mochila" => 99.00
]; // vetor não ordenado chave => valor

foreach ($precos as $produto => $preco){
    echo "$produto: R$ number_format($preço,2)";
}
```

Desafio : Simuladro de Cobrança (FINANSENAI)

Desafio Final

Semana 4 - Modularização com Funções

Principio do DRY (`Don´t Repeat) Yourself`)

Se uma lógica foi escrita duas vezes ou mais dentro de um código, essa lógica deve virar uma função.

Funções Nativas do PHP

O PHPH tem milhares de funções prontas, essa funç~eos são chamadas de nativas.

- **O que é uma Função?**

Uma função é como uma máquina: você coloca uma matéria-prima(Parâmetro), ela processa e devolve um produto final(Retorno)

Exemplo de Função Nativa:

```php
$texto = "senai americana";

//str_replace(le abusca um pedaço do texto e substitui por outro)
$textoNovo = str_replace("americana","são paulo",$texto);

//strtoupper
echo strtoupper($textoNovo); // SENAI SÃO PAULO
```

Principais Funções Nativas (Mais Utilizadas)

As funções abaixo já fazem parte do PHP e podem ser chamadas diretamente no código. Observe os parâmetros que cada uma recebe e o tipo de informação que ela retorna.

Função	Categoria	O que faz	Como usar
`strlen()`	Strings	Retorna a quantidade de caracteres de um texto.	`$tamanho = strlen($texto);`
`strtoupper()`	Strings	Converte o texto para letras maiúsculas.	`$resultado = strtoupper($texto);`
`strtolower()`	Strings	Converte o texto para letras minúsculas.	`$resultado = strtolower($texto);`
`ucfirst()`	Strings	Converte a primeira letra do texto para maiúscula.	`$resultado = ucfirst($texto);`
`trim()`	Strings	Remove espaços e quebras de linha no início e no fim do texto.	`$limpo = trim($texto);`
`str_replace()`	Strings	Substitui uma parte do texto por outra.	`$novo = str_replace("-", "", $cpf);`
`substr()`	Strings	Extrai uma parte do texto a partir de uma posição.	`$inicio = substr($texto, 0, 3);`

Função	Categoria	O que faz	Como usar
`explode()`	Strings	Divide um texto e cria um array usando um separador.	`$palavras = explode(" ", $nome);`
`implode()`	Arrays	Junta os itens de um array em um único texto.	`$lista = implode(", ", $nomes);`
`count()`	Arrays	Conta a quantidade de itens de um array.	`$total = count($produtos);`
`in_array()`	Arrays	Verifica se um valor existe dentro de um array.	`$existe = in_array("SP", $estados, true);`
`array_push()`	Arrays	Adiciona um ou mais itens ao final de um array.	`array_push($nomes, "Ana");`
`array_pop()`	Arrays	Remove e retorna o último item de um array.	`$ultimo = array_pop($nomes);`
`sort()`	Arrays	Ordena um array em ordem crescente e reorganiza suas chaves.	`sort($notas);`
`array_keys()`	Arrays	Retorna um array contendo as chaves de outro array.	`$chaves = array_keys($produtos);`
`number_format()`	Números	Formata um número com casas decimais e separadores definidos.	`$preco = number_format($valor, 2, ',', '.');`
`round()`	Números	Arredonda um número para a quantidade de casas informada.	`$media = round($nota, 2);`
`max()`	Números	Retorna o maior valor de uma lista ou array.	`$maior = max($notas);`
`min()`	Números	Retorna o menor valor de uma lista ou array.	`$menor = min($notas);`
`is_numeric()`	Validação	Verifica se o valor é um número ou uma string numérica.	`if (is_numeric($entrada)) { ... }`
`isset()`	Validação	Verifica se uma variável existe e não possui valor `null`.	`if (isset($usuario)) { ... }`
`empty()`	Validação	Verifica se uma variável está vazia.	`if (empty($pedido)) { ... }`
`date()`	Data e hora	Formata uma data ou hora conforme uma máscara.	`$hoje = date('d/m/Y');`
`file_exists()`	Arquivos	Verifica se um arquivo ou diretório existe.	`if (file_exists('dados.txt')) { ... }`
`file_get_contents()`	Arquivos	Lê todo o conteúdo de um arquivo ou endereço.	`$conteudo = file_get_contents('dados.txt');`
`file_put_contents()`	Arquivos	Grava conteúdo em um arquivo, criando-o se necessário.	`file_put_contents('log.txt', $mensagem);`

Atenção: algumas funções modificam o array original, como `sort()`, `array_push()` e `array_pop()`. Já outras retornam um novo valor, como `count()`, `explode()` e `str_replace()`. Em caso de dúvida, consulte a documentação oficial do PHP e verifique o retorno da função.

Documentação PHP

Acesse a documentação oficial do PHP em português

Consulte também a referência de funções do PHP para pesquisar a sintaxe, os parâmetros eos valores por cada função.

Funções Customizadas (Criando suas próprias máquinas)

Quando o PHP não tem a função que queremos, nós a criamos!

A Regra de Ouro: Uma função deve focar em `return`(retornar um valor), e não imprimir (`echo`).

Veja a diferença nesse exemplo:

```php
function calcularTotal($preco, $quantidade){
    //a função calcula e rotna o resultado, mas não imprime nada
    return $preco * $quantidade;
}

$total = calcularTotal(25.00, 3);

echo "Total da compra: R$ " . number_format($total, 2, ",", ".");
// Total da compra: R$ 75,00
```

A função `calcularTotal()`pode ser reutilizada em uma página, relatório ou teste. O `echo`aparece somente fora da função, no momento de apresentar o resultado ao usuário

Padrão de Uso Corporativo (PHP 8 Strict Types)

No merdado de trabalho, exigimos que a função avise extamente o **TIPO** de dado que ela espera receber e o **TIPO** que ela vai devolver.

Isso é chamado de **tipagem de funções**. Ao declara os tipos, o código fica mais fácil de entender e o PHP consegue identificar alguns erros antes que eles causem problemas maiores no sistema.

Os tipos mais usados:

- `int`: número inteiro, `10` ou `1024`.
- `float`: número decimal ou ponto flutuante, `10.50`.
- `string`: texto, como `"Maria"`
- `bool`: valor lógico, `true` ou `false`.
- `void`: identifica que a função não devolve nenhum valor

O tipo deve ser escrito antes do nome de cada parâmetro e o tipo da função deve ser escrito após os parênteses, precedito po `:`, informando o que a função vai devolver.

Exemplo de uso de função e parâmetros tipados:

```php
function apresentarProduto(string $nome, float $preco): string{
    return "$nome custa R$ $preco";
```

```
}

$mensagem = apresentaproduto("Caderno", 25.90);
echo $mensagem;
// Caderno custa R$ 25.90
```

> **Resumo**: os tipos dos parâmetros documetam as entradas da função, o tipo após `:` documeta a saída da função

O Tipo Mágico : `void`

Se uma função faz um trabalho interno e **não retrona NADA**, dizemos que o retorno dela é "vazio" (`void`).

Exemplo de função sem retorno:

```
function registroLog(string $mensagem): void{
    //apenas salvar em um arquivo de texto, não devolver nenhuma variável
    file_put_contents("erro.log",$mensagem);
}
```

Escopo e Referência (O segredo da memória)

O que é Escopo? (A Regra de Las Vegas)

O que acontece dentro da função, fica dentro da função. Uma variável criada fora nã existe lá dentro, e uma criada lá dentro morre quando a função acaba.

Escopo é o local do programa onde a variável pode ser armazenada/acessada. Em PHP, uma variável criada fora de uma função pertende ao **escopo global**. uma variável criada dentro de uma função pertence ao **escopo local**.

Exemplo de Escopo de variável:

```
$nomeSistema = "CRM Senai"; //Variável global

function criarMensagem():string{
    $mensagem = "Bem-Vindo!"; //Variável local
    return $mensagem;
}

echo $nomeSistema; //Correto: esta no escopo global
echo criarMensagem(); //Correto: a função devolve sua variável local.
//echo $mensagem; // Incorreto: $mensagem só existe dentro da função, não é acessada fora
```

- Como enviar dados para uma função?

A forma mais segura e organizada é enviar os dados por **parâmetros**. Assim, a função não precisa acessar diretamento variáveis globais:

```
function saudar(string $nome):string{
    return "Olá, $nome!";
}
```

```php
$nomeCliente = "João";
echo saudar($nomeCliente); // Olá, João!
```

Nesse caso, `$nomeCliente` continua no escopo global, mas seu valor é enviado para o parâmetro local `$nome`. A função recebe uma informação, processa e retorna o resultado.

Exemplo Incorreto:

```php
$nome = "João";
function saudar():string{
    return "Olá, $nome";
}
```

A função `saudar()`não conhece a variável globla `$nome`

> **Resumo:** variáveis protegem os dados internos da função; parâmetros são o caminho recomendado para evitar Erros e enviar Informações, e `return`é usado para devolver um resultado ao código que chamou a função.

Semana 5 - Arrays e Manipulação Avançada de Dados

Um array(também conhecido como vetor) é uma estrutura de dados usadas para armazenar vários valores em uma única variável.

Tipos de Arrays em PHP:

- Indexados/Ordenado(Númerica): Usam Números inteiros como índices(chaves), que começam em zero por padrão;
- Associativos/NãoOrdenados(String): Usam chaves(String) para identificar valores;
- Multidimensionais: Contêm um ou mais arrays dentro de outro array.

Exemplos de Arrays:

```php
//array indexado
$frutas = ["maça", "banana", "laranja"];

//array associativo
$capitais = [
    "SP" => "São Paulo",
    "RJ" => "Rio de Janeiro",
    "MG" => "Belo Horizonte",
    "ES" => "Vitória",
];

//acessando os dados dos Arrays

echo $frutas[1]; // banana
echo $capitais["MG"]; //Belo Horizonte
```

> Obs: Em arrays associativos, nos trocamos os nº do índice por Nomes(Chaves/Keys). Na Delaração do Vetor usamos setinha(=>) que significa "recebe"

Arrays Multidimensionais (Banco de Dados na Memória)

É aqui que o "BackEnd" começa de verdade. o Array Multidimensional é o formato como os Bancos de Dados e Apis respondem as solicitações feitas pelo BackEnd.

Exemplo de Array Multidimensional:

```
$clientes = [
    ["id" => 1, "nome" => "Ana", "email" => "ana@email.com", "ativo" => true],
    ["id" => 2, "nome" => "Bruno", "email" => "bruno@gmail.com", "ativo" => false],
    ["id" => 3, "nome" => "Carlos", "email" => "calos@hotmail.com", "ativo" => true],
];

//Como Acessar o Email do Carlos
echo $clientes[2]["email"]; // carlos@hotmail.com
```

O Melhor amigo dos Array: `O Foreach`

O laço de repetição especial para arrays. O `foreach` percorre cada elementos de um array

Exemplo de Aplicação:

```
foreach($clientes as $clienteAtual){
    echo $clienteAtual["nome"];
    echo $clienteAtual["email"];
}
// vai imprimir nome e email de todos os Clientes do Array
```

Transformação de Arrays e Arrow Function

Transformações de arrays são usadas para modificar ou filtrar informações de um array existente

- `array_filter` Serve para buscar dados em um array e devolver apenas os dados que passarem pelo filtro

```
$clientesAtivos = array_filter($clientes, fn($c) => $c["ativo"]===true);
//novo array , tera apenas os clientes que a chave ativo for igual a true
```

- `array_map` Serve para alterar Todos os dados de um array de uma única vez

```
$produtos = [
    ["id"=>1, "preco"=10.00, "setor"=>"jardim"],
    ["id"=>2, "preco"=15.90, "setor"=>"ferramenta"],
    ["id"=>3, "preco"=23.50, "setor"=>"jardim"],
]
//ajustar o preço de todos os produtos em 10% de aumento

$produtosAjustados = array_map(fn($p) => $p["preco"] = $p["preco"]*1.1, $produtos);
```

> Obs: para a função de filtragem, primeiro selecionamos a array e depois criamos a função de filtro. Para a função de mapeamento, primeiro criamos a função de transformação e depois aplicamos no array.

Debugando um Array (Kit de PRimeiros Socorros)

- `print_r` função usada para exibir informações sobre um array de forma legível em liguagem natural

```
echo print_r($frutas);
//array
(
    [0] => "maça",
    [1] => "banana",
    [2] => "laranja"
)
```

- `var_dump` Exibi com mais detalhes as informações de um array ou variável em PHP

```
echo var_dump($frutas);
// Mostrar Tudo: tipo de dados, o tamanho e o valor
```

Semana 6 - Processamento HTTP e Formulários Web

Anatomia de um Formulário HTML para BackEnd

Antes do PHP processar qualquer informação, precisamos coletar informações no FrontEnd através de um `<form>`

Exemplo de `<form>` HTML

```
<form action="processar.php" method="POST">
    <label>Nome Completo</label>
    <input type:"text" id="campoNome" name="nomeUsuario" placeholder="Digite seu Nome">
    <button type="submit">Cadastrar</button>
</form>
```

Os 3 Pilares de um formulário

1. action="processa.php" -> O Destino : Define qual script PHP no servidor recebrá os dados
2. method="POST" -> O Transporte: Define a via de protocolo HTTP que será usada (GET ou POST)
3. name="nomeUsuario" -> A Etiqueta do Dado: É o nome da chave que o PHP usará no array associativo ($_POST["nomeUsuario"])

> obs: Nunca Confundir `id`com `name`no input, o PHP ignora o `id`.

O Protocolo HTTP

Quando o usuário clica no botão `type="submit"`, o navegador compila todas as informações dos campos preenchido e dispara um pacote de comunicação padronizado pelo **Protocolo HTTP(Hypertext Transfer Protocol)**.

Os Formatos de Transferência

- **Método GET**: solcitar informações públicas e realizr buscas, mas altamente arriscado para dados privados.
- **Método POST**: As informações viajam guardadas dentro do protocolo.

Testar o uso dos Protocolos HTTP

OK

GET vs. POST

1. O Método GET(Consultas e Filtros)

O método `GET`é utilizado quando a intenção do cliente é **buscar ou filtrar dados** sem alterar o estado do servidor. Os dados enviados via `GET` são anexados diretamente ao final da URL na forma de uma **Query String**

2. O Método POST (Envio de Cargas Úteis e Mutações)

O método `POST` é utilizado quando o formulário envia dados que devem ser processados para **criar ou modificar registros** no sistema (Ex: Cadastro de usuários, finalizações de compra, upload de arquivos)

As SuperGlobais

As Variáveis SuperGlobais são arrays intrnos pré-definidos que estão sempre acessíveis em qualquer parte do script php, sem precisar declarar.

- **$_GET**: Armazena dados passados pela URL via parêmetros de consulta(query string);
- **$_POST**: Recolhe dados enviados por formulários usando método HTTP POST.
- **$_SERVER**: Contém informações sobre o servidor, ambiente e caminhos de script

Porque usar `??` para obter dados da SuperGlobal?

Usamos o Operador de Nulidade (Coalescência Nula) para verificar se o valor da variável não é `null`, se caso for `null` atribuimos um outro valor para evitar erros no script.

Exemplo de Uso:

Na primeira vez que uma página é aberta, o formulário ainda não foi enviado. Portanto, a achave pode não existir no array.

```php
$nome = $_POST["nome"];
// escrevendo desta forma, o código pode gerar um aviso de erro.

// a forma mais correta de escrita é
$nome = $_POST["nome"] ?? "";
//Se $_POST["nome"] não existir, use uma string vazia.

//outra forma de verificar nulidade é usando if else
if(isset($_POST["nome"])){
    $nome = $_POST["nome"];
} else{
    $nome = "";
}
```

> obs: use htmlspecialchars() ao exibir valor em HTML => converte caracteres especiais em entidades correspondentes em HTML, evitando que o código seja interpretado erradamente pelo navegador. É usado principalmente na segurança web para evitar ataques Cross-Site-Scripting(XSS).

Validação de Dados no BACKEnd é Obrigatória.

Muitos desenvolvedores iniciantes acreditam que colocar atributos `required`, `type=email`ou `min=0`na do HTML é suficiente para proteger o sistema. **Isso é uma ilusão!**. Sempre fazer as validações de dados no código BackEnd

Funções Nativas Essenciais para Limpeza e Validação de Dados

A validação no Back-End deve acontecer sempre antes do processamento de qualquer dado recebido pelo usuário. Abaixo está uma tabela resumida das funções nativas do PHP usadas com frequência para limpar, verificar e validar entradas de formulário.

Função	Descrição	Quando usar	Exemplo simples
`trim($valor)`	Remove espaços no início e no fim da string	Limpar texto digitado pelo usuário	`$nome = trim($_POST['nome'] ?? '');`
`htmlspecialchars($valor, ENT_QUOTES, 'UTF-8')`	Converte caracteres especiais em entidades HTML seguras	Exibir dados na tela sem risco de XSS	`echo htmlspecialchars($_POST['nome'] ?? '', ENT_QUOTES, 'UTF-8');`
`filter_var($valor, FILTER_VALIDATE_EMAIL)`	Valida formato de e-mail	Campos de e-mail	`filter_var($email, FILTER_VALIDATE_EMAIL)`
`filter_var($valor, FILTER_VALIDATE_INT)`	Verifica se o valor é inteiro válido	Idade, código, quantidade	`filter_var($_POST['idade'] ?? '', FILTER_VALIDATE_INT)`
`filter_var($valor, FILTER_VALIDATE_FLOAT)`	Verifica se o valor é número decimal válido	Preço, peso, altura, salário	`filter_var($_POST['preco'] ?? '', FILTER_VALIDATE_FLOAT)`
`isset($variavel)`	Verifica se uma variável existe e não é `null`	Garantir que o campo foi enviado	`if (isset($_POST['nome'])) { ... }`
`empty($valor)`	Verifica se o valor está vazio	Campos obrigatórios	`if (empty($_POST['senha'])) { ... }`
`strlen($valor)`	Retorna o tamanho da string	Exigir mínimo ou máximo de caracteres	`if (strlen($senha) < 6) { ... }`
`in_array($valor, $lista, true)`	Verifica se o valor pertence a uma lista permitida	`select`, `radio`, opções válidas	`in_array($categoria, ['A','B','C'], true)`
`is_numeric($valor)`	Confirma se o valor é numérico	Validação de número	`if (is_numeric($_POST['quantidade'])) { ... }`
`preg_match($padrao, $valor)`	Valida por expressão regular	CPF, CEP, telefone, senha forte	`preg_match('/^\d{5}-\d{3}$/', $cep)`
`filter_input(INPUT_POST, 'campo', FILTER_SANITIZE_SPECIAL_CHARS)`	Captura e limpa dados da requisição	Ler entradas com segurança	`$nome = filter_input(INPUT_POST, 'nome', FILTER_SANITIZE_SPECIAL_CHARS);`

Preservação de Estado em Formulários (*Sticky Form*)

A técnica do **Sticky Form** consiste em imprimir de volta no atributo "value" do input os dados que o usuário acaba de digitar caso ocorra um erro de validação de dados.

Exemplo de Uso:

```
<div class="campo">
    <label for="nome">Nome Completo</label>
    <input type="text" id="nome" name="nome"
        value="<?= htmlspecialchars($dadosFormulario["nome"] ?? "") ?>
        class="<?= isset($erro["nome"]) ? "input-erro" : "" ?>">
    <?php if (isset($erro["nome"])): ?>
        <span class="erro-texto"><?= $erro["nome"] ?></span>
    <?php endif; ?>
</div>
```

Semana 7 - Segurança no BackEnd - Sanitização, Validação e Proteção contra XSS

1º Mandamento do Desenvolvedor BackEnd -

> Nunca Confie no Usuário : Toda entrada de dados vindo de fora do servidor é potenciamente maliciosa até que seja rigorosamente validada, sanitizada e codificada.

Quando você disponibiliza um campo de texto em um site, qualquer pessoa conectada a internet pode digitar códigos maliciosos em vez de texto. Se o código BackEnd pega esse texto diretamente sem nenhum tratamento, a ordem de execução de códigos abrirá porta para a invasão devastadoras do seu sistema.

A Anatomia de um Ataque: O que é Cross-Site Scripting (XSS)

O XSS ocorre quando uma aplicação web inclui dados não confiaveis em uma página web sem a devida validação ou escape de caracteres. Isso permite que um atacante execute scripts maliciosos(geralmente em javaScript) diretamente no navegador de outro usuário que visitam o site.

As Principais Modalidade de Ataques:

1. *Roubo de Sessão(Cookie Stealing)*: O JavaScript injetado lê os cookies de autenticação da vítima (documente.cookie) e os envia para o servidor do atacante , permitindo que ele faça login na conta da vítima sem precisar de senha.
2. *Desconfiguração do Site(Defacement)*: Alterar visualmente o site, inserindo mensagens falsas, banners ofensivos ou formulários de login fraudulentos (phising interno).
3. *Redirecionamento Malicioso*: Força o navegador da víima a abrir sites com vírus ou páginas clonadas de banco.
4. *Captura de Teclas(Keylogger)*: Grava tudo o que a vítima digita enquanto a página estiver aberta.

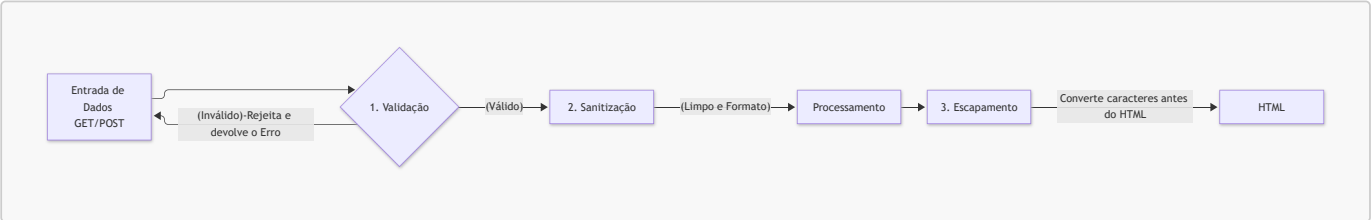
Os Vetores de Ataques Mais Frequentes:

Nem todo ataque XSS usa a tag óbvia `<script>`. Desenvolvedores que tentam bloquear XSS apenas "apagando a palavra script" são facilmente burlados por atacantes:

Vetor de Injeção	Como funciona o ataque?
`<script>alert('XSS')</script>`	Injeção direta de bloco de script executável pelo navegador.

Vetor de Injeção	Como funciona o ataque?
`<img src="invalido.jpg" onerror="alert('XSS')">`	O navegador tenta carregar a imagem inexistente e dispara o evento `onerror` com o JavaScript.
`<svg onload="alert('XSS')">`	O navegador renderiza o elemento gráfico SVG e executa o evento `onload`.
`<a href="javascript:alert('XSS')">Clique</a>`	O clique no link executa a pseudo-URL com JavaScript em vez de abrir um site.
`"><script>alert('XSS')</script>`	Usado quando o dado é impresso dentro de um `<input value="...">`, quebrando o atributo e injetando a tag.

A Tríade da Defesa: Validação, Sanitização e Escapamento



1. **Validação**: Verifica se o dado recebido atende aos requisitos exatos do sistema (tipo, tamanho, formato).

Ex: Verificar se o e-mail possui `@` e dominio válido (`filter_var($email, FILTER_VALIDATE_EMAIL)`).

2. **Sanitização**: Transforma o dado para adequa-lo ao formato desejado, removendo caracteres indesejados

Ex: Remover espaços no ínicio e fim (`trim($nome)`)

3. **Escapamento/Codificação de Saída**: é o ato de converter caracteres especiais de linguagem HTML em suas respectivas **Entidades HTML** no momento exato em que eles são impressos na tela.

Ex: usar `htmlspecialchars()`

A Ferramenta PRincipal: `htmlspecialchars()`

A função `htmlspecialchars()` é o principal mecanismo do PHP para neutralizar XSS na camda de apresentação

Como a conversão de entidades funciona?

Caractere Original	Entidade HTML Gerada	Efeito no Navegador
`<`	`<` (*Less Than*)	O navegador exibe `<` na tela, mas **não cria uma tag**.
`>`	`>` (*Greater Than*)	O navegador exibe `>` na tela sem fechar tags.
`"`	`"` (*Quotation Mark*)	Não quebra atributos HTML `<input value="...">`.
`'`	`'` ou `'`	Protege strings envoltas em aspas simples.
`&`	`&` (*Ampersand*)	Evita interpretação incorreta de entidades.

A Sintaxe no PHP

```
string htmlspecialchars(
    string $string,
    int $flags = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML5,
```

```
    ?string $encoding = "UTF-8"
)
```

- `ENT_QUOTES`: Converte tatos aspas duplas quanto aspas simples. Essencial para saídas em atributos HTML
- `ENT_SUBSTITUTE`: Substitui sequências de bytes inválidos por caracteres de substituição Unicode em vez de retornar uma string vazia
- `ENT_HTML5`: Aplica a tabela de entidades compatíveis com a especificação HTML5
- `UTF-8`: Garante que caracteres da lingia portuguesa (como "ç", "ã", "é") sejam preservados sem corrupção

A função Helper de Escapamento

Para não precisar digitar essa linha extensa em todas as partes de saída de texto para HTML, os desenvolvedores profissionais criam uma função auxiliar curta:

```
function e(string $texto):string {
    return htmlspecialchars($texto, ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML5, "UTF-8");
}

<p>Comentário: <?= e($comentarioUsuario) ?></p>
<input type="text" name="nome" value="<? e($nomeUsuario) ?>"/>
```

Validação e Sanitização com `filter_var()`

O PHP possui a biblioteca de filtros nativos `filter_var()`. Observe os filtros mais importantes do ecossistema corporativo:

```
<?php
declare(strict_types=1);

// 1. Validação de E-mail
$email = "usuario.teste@senai.br";
if (filter_var($email, FILTER_VALIDATE_EMAIL) !== false) {
    // E-mail válido
}

// 2. Validação de Número Inteiro com Limites (Range)
$idade = "25";
$opcoesIdade = [
    'options' => [
        'min_range' => 16,
        'max_range' => 120
    ]
];
if (filter_var($idade, FILTER_VALIDATE_INT, $opcoesIdade) !== false) {
    // Idade é um inteiro entre 16 e 120
}

// 3. Validação de URLs (Links)
$website = "https://www.sp.senai.br";
if (filter_var($website, FILTER_VALIDATE_URL) !== false) {
    // URL possui protocolo e formato válidos
}

// 4. Validação de Endereço IP
$ip = "192.168.1.100";
if (filter_var($ip, FILTER_VALIDATE_IP) !== false) {
```

```
    // IP válido
}
```

Semana 8 - Persistência de Dados com Banco de Dados Relacionais (PostgreSQL) e Conexão PDO

Tema: Camada de acesso a Dados, DriverPDO(PHP Data Objects), Driver `pdo_pgsql`, Padrão Singleton, Isolamento de Credenciais(`.env .ini`) e Tratamento de Exceções(`PDOException`)

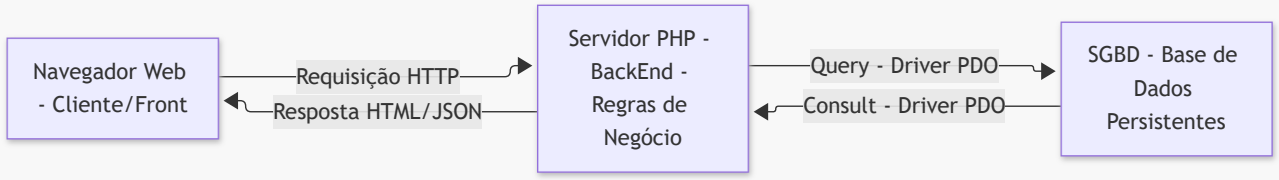
1. Da Memória Volátil ao Banco de Dados

Em sistemas corporativos de grande porte, arquivos planos (`.txt .json`) não oferecem a segurança , integridade, concorrencia e velocidade necessária para armazenamentos de dados. Então é aqui que o **BackEnd** encontra o **Banco de Dados Relacional**.

Banco de Dados Relacional Permite:

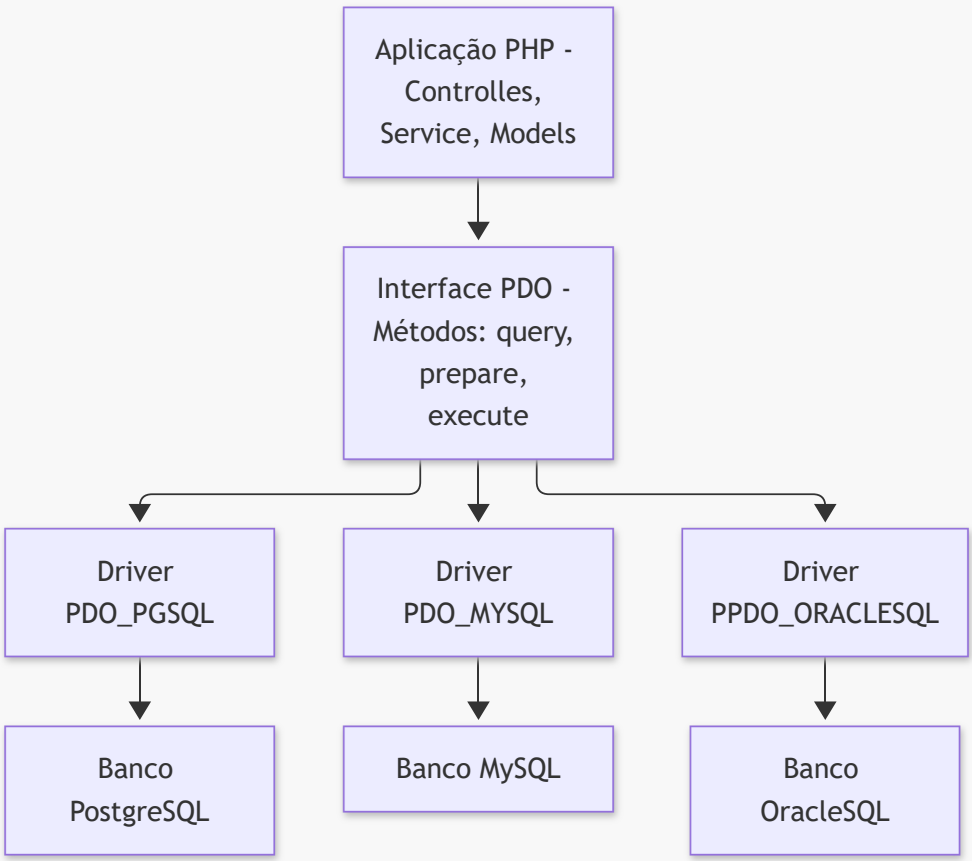
- Conectar a lógica de programação server-side ao Sistema de Gerenciamento de Banco de Dados (SGBD).
- Garantindo persistÊncia definitiva e segura dos registros.
- Aplicando integridade referencial, constraints, consultas otimizadas e produtividade ACID aprendidas na disciplina de Banco de Dados.

> obs: ACID: Atomicidade, assegura que cada transação seja única. Consistência, respeita todas as regras, restrições e chaves definidas, garantindo a validade da transação. Isolamento, transações são realizadas de forma independente. Durabilidade, transações são confirmadas, garantindo persistência permanente.



2. O que é o PDO(PHP Data Object)?

O **PDO** é uma camaada de abstração de acesso a dados integrada nativamente ao PHP. Ele fornece uma interface uniforme e orientada a objetos para se comunicar com múltiplos sistemas de banco de dados (PostgreSQL, MySQL, SQLite, OracleSQL, SQLServer).



3. Vantagens do uso do PDO

- **Portabilidade de Código**: Os métodos de conexão, consulta e transções são identicos, independente do banco utilizado. Se o cliente migrar de banco Postgres para outrro SGBD(MySQL), o programador apenas altera a string DSN de conexão, preservando toda a lógica de acesso já utilizada ou criada.
- **Suporte Nativo a Prepared Statement**: O PDO foi projetado para trabalhar com consultas nativas, oferencendo defesa contra ataques de **SQL_Injection**.
- **Tratamento Orientado a Objetos com Exception**: Em vez de retronar códigos de erros, o PDO lanca uma instancia da classe especializada `PDOExcepton`.

A Sintaxe da Conexão PDO: DSN(Data Source Name)

Para que o PDO saiba onde o banco está localizado, em qual porta abrir, utilizamos a string padronizada **DSN**.

```
pgsql:host=127.0.0.1;port=5432;dbname=seu_banco
  |          |           |           |
  |          |           |           └─ Nome da base de dados ralacional(nome do banco)
  |          |           └─ Porta padrão do Banco de Dados PostgreSQL(5432)
  |          └─ Endereço IP ou hostname do servidor
  └─ Identificador do driver do SGBD(pgsql) - PostgreSQL
```

4. A Configuração do PDO

Ao instanciar um objeto PDO, devemos configurar quadro flags essenciais que determinam como o driver de comportará frente a erro e consultas ao SGBD

```
$opcoes = [
    //1. flag: Lança exceções imediatamente quando ocorrer qualquer erro SQL
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,

    //2. Retorna registros apenas com nomes das colunas (Eliminar duplicidade numérica)
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,

    //3. Desatica emulação e utiliza prepared statements nativos
    PDO:: ATTR_EMULATE_PREPARES => false,

    //4. Limita a 5 segundos para tentar a conexão com o servidor do BD
    PDO:: ATTR_TIMEOUT => 5
];
```

Detalhamento das Flags:

- PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION : por padrão o PDO pode falhar silenciosamente e retorna apenas `false`. Ao Ativar o ERR_MODE força o PHP a dispara uma `PDOException`, permitindo que o nosso código interprete qualquer erro em um bloco `try-catch`.
- PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC : por padrão o métos `fetch()`retrona um array duplicado ontendo índices numéricos`[0,1]`e associativos`["id","código_maquina"]`. Definir `FETCH_ASSOC`reduz o consumo de memória RAM pela metade e entrega coleções limpas.
- PDO::ATTR_EMULATE_PREPARES => false : Garante que o PHP envie a consulta e os parâmetros separados diretamente para o planejador do BD processar, blindando e aplicação contra ataques sofisticados de `SQL_injection`

5. Proteção de Credenciais

Um dos erros mais graves cometidos por desenvolvedores iniciantes é escrever dados de conexão diretamente dentro do código:

```
//péssima prática de código
$pdo = new PDO("pgsql:host=localhost; dbname="producao"; "postgres"; "senha12345");
//observer que as credenciasi estão expostas nos código
```

Se esse arquivo for versionado e enviado para GitHub:

1. Suas senhas de produção ficam públicas
2. Robôs maliciosos varrem repositórios à procura de credenciais expostas, para invadir banco de dados e sequestrar informações(ataque de Ransoware)
3. A empresa é penalizada por violações da **LGPD(LEi Geral de Proteção de Dados)**

A Abordagem Segura: Usando Arquivos de Configuração Isolada (`.ini .env`)

Isolamos as credenciais em um arquivo externo protegido que **nunca entra no Git**

```
; config/database.ini
[database]
db_driver   = pgsql
db_host     = 127.0.0.1
db_port     = 5432
db_name     = producao
```

```
db_user     = postgres
db_pass     = senha12345
```

Adiconameos o Arquivo Isolado ao `.gitignore`

```
config/database.ini
.env
logs/*.log
```

---

6. Padrão Singleton de Conexão

Imagian uma aplicação web com 500 usuários acessando simultaneamente. Se cada script, função ou método executar `new PDO()`, ou seja, abrir uma nova conexão, sempre que precisar consultar o banco de dados, teremos milhares de conexão de redes abertas desnecessariamente.

No SGBD(PostgreSQL), cada conexão aberta cria um processo no sistema operacional dedicado. Abrir conexões repetidas esgotam rapidamente o limite configurado (`max_connection`) do BD gerando um erro: `Fatal Error: sorrym, too many clients already`

Como o Singleton Resolve Isso

O pdrão **Singleton** garante que **apenas uma única instancia de conexão PDO exista por requisição**, reutilizando a conexão existente em qualquer ponto do sistema.

As Configurações do Singleton

1. **Construtores Privados** (`private function _constructor`): Impede que outros arquivos instanciem uma nova conexão
2. **Propriedade/Atributos Estáticas Privadas**: (`private static ?PDO $instancia = null`): Aramzena a Conexão aberta na Classe
3. **Métodos de acesso Estáticos Públicos**: (`public static function obterConexao():PDO`): A Conexão é criada pelo método, garantindo acesso a conexão, mas não acesso aos atributos da conexão, se caso já existir uma conexão, apenas devolve a conexão existente para o operador, sem a necessidade de crir uma nova.
4. **Bloqueio de Clonagem e Desserialização**: (`_clone` e `_wakeup`): Garantir que ninguém consiga duplicar o objeto da conexão.

7. Tratamento de Falhas com `PDOException`

Quando uma tentativa de conexão falha(servidor desligado, senha incorreta, porta inacessível ...), o PDO lança uma Exceção (`PDOException`). Então, devemos tratar essa falhas.

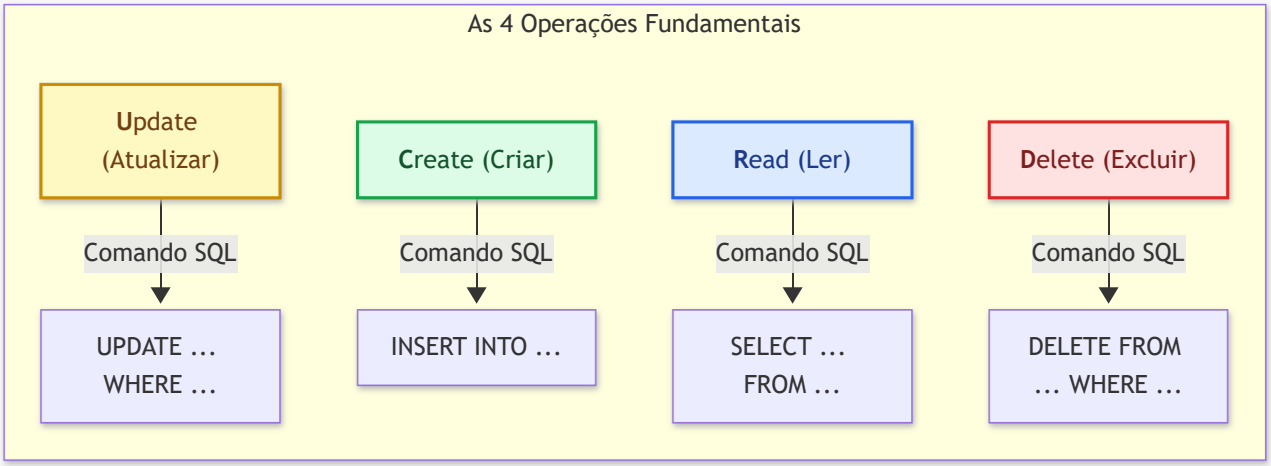
Práticas recomendadas de segurança (AppSec):

- **Para o Usuário**: Exibir mensagens amigáveis e genéricas: *Não é possível processar sua solicitação. Tente novamente mais tarde*;
- **Para a Equipe de Desenvolvimento**: Gravar os detalhes técnicos da falha com timestamp(carimbo de data e hora) em um arquivo de log seguro (`log/database.log`);

Semana 9 - CRUD Completo com Prepared Statements e Proteção Contra SQL Injection

Tema: Operações CRUD, Vulnerabilidade SQL Injection (OWASP Top 10), Consultas Preparadas com PDO (`prepare`, `bindValue`, `execute`), Marcadores Nomeados e Padrão de Arquiteutra DAO (DATA ACCESS OBJECT)

Em qualquer organização , o objetivo central de um sistema de software é manipular informações com segurança, velocidade e consistência. Essa Manipulação se resuma a quatro operações fundamentis que todo desenvolvedor BackEnd deve dominar com perfeição, essa operações são conhecidas pelo acrônimo **CRUD**.



Na Semana 08 , aprendemos a como conectar usando a extensão PDO e utilizando o padrão Singleton. Agora, vamos dar vidar a essa conexão: Aprenderemos a inserir novos registros, consultar com filtros dinâmicos e remover e atulizar dados com segurança.

A Maior Ameaça da História da Web: SQL Injection (`SQLi`)

Antes de Escrevermos a primeira query de manipulação, precisamos compreender o perido que cerca o acesso a banco de dados.

A vulnerabilidade **SQL Injection** que ocupa o topo das listas mais críticas de cibersegurança. Ela Ocorre quando um desenvolvedor comete um erro gravíssimo de **concatenar entradas fornecidas pelo usuário diretamente na instrução SQL**

Exemplo de Código Proibido (Concatenação de String)

Imagina um sistema que valida o login de um operador da seguinte forma:

```
//Codigo Vulnerável e Perigoso - NUNCA FAÇA ISSO!
$usuario = $_POST["usuario"];
$senha = $_POST["senha"];
// o invasor digita no campo usuário: admin' --

$sql = "SELECT * FROM usuarios WHERE login = '".$usuario . "'AND senha= '" .$senha ."'";
$resultado = $pdo->query($sql)
```

O que acontece quando o atacante digita: `admin' --`?

A string final mpontada pelo PHP envia seguinte mensagem para o banco

```
SELECT * FROM usuarios WHERE login = 'admin' --' AND senha= '...'
```

1. A aspa digita pelo atacante fecha a string do login antecipadamente
2. O operador `--`no Banco de Dados indica o **início de um comentário**
3. O restante da query (a validação de senha) é ignorada pelo morto de busca do banco

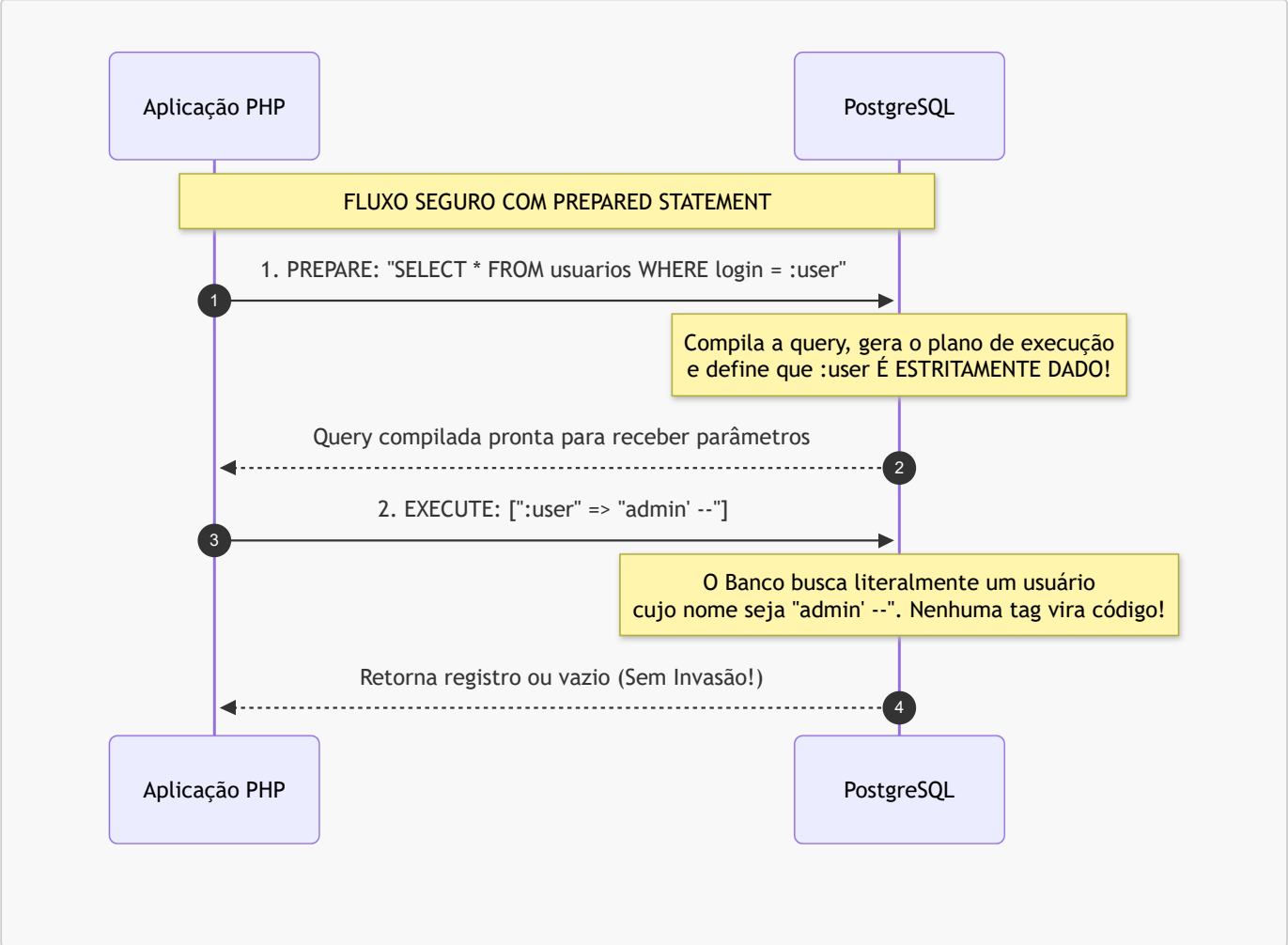
4. **Resultado:** o invasor faz login instantaneamente na conta do administrador sem precisar saber a senha.

3 Cenários Mais utilizados de SQL Injection

Tipo de Injeção	Payload Injetado pelo Invasor	Consequência no PostgreSQL
Bypass de Autenticação	`' OR '1'='1`	A condição torna-se sempre verdadeira, retornando o primeiro usuário da tabela (geralmente o administrador do sistema).
Exfiltração de Dados (UNION SQLi)	`' UNION SELECT id, nome, senha FROM usuarios --`	O invasor anexa tabelas sigilosas inteiras no resultado da consulta visível na tela, violando a LGPD.
Destruição / Adulteração (Stack Queries)	`'; DROP TABLE pecas_industriais; --`	Dependendo do driver e das permissões do usuário, o invasor encerra a consulta atual e executa comandos de destruição em massa.

A Defesa Definitiva: PREPARED STATEMENTS

Usar consulta com Prepared Statements evita que ocorra SQL Injection



Por que a consulta preparada (`prepare`) é imune a injeções?

Quando utilizamos `$pdo->prepare()`, ocorre uma **separação física e temporal** entre o **comando SQL** e os **dados do usuário**:

1. **Fase de Compilação(`prepare`)**: O BAnco recebe o molde da instrução com marcadores (`:parametros`). Ele analisa a sintaxe, otimiza o caminho de busca e compila o plano de execução. A estrutura lógica da consulta está **fechada e congelada**

2. **Fase de Envio dos Dados (`execute`)**: O PHP envia apenas os valores literais. Memso que o invasor envie aspas, ponto-e-vírgula ou comandos `DROP`, o Banco tratará tudo exclusivamente como um texto inofensivo pertencente aquele campo.

Marcadores Nomeados e Posicionais no PDO

O PDO aceita dois formatod de marcadores em prepared statements

1. **Marcador Posicional (`?`):**

```
//Funcionam, mas é sujeito a erros de contagem de parâmetros em queries longas
$sql = "INSERT INTO usuarios (id, nome, email, telefone) VALUES (?, ?, ?, ?)";
$stmt = $pdo->prepare($sql);
$stmt->execute([$id, $nome, $email, $telefone]);
```

2. **Marcadores Nomeados (`:nome`) - Padrão Recomendado**

```
// Autenticação Autoexplicativa, altamente legível e a prova de inversão de ordem
$sql = "INSERT INTO usuarios (id, nome, email, telefone) VALUE (:id, :nome, :email,
:telefone)";
$stmt = $pdo->prepare($sql);
$stmt->execute([
    ":id"       => $id,
    ":nome"     => $nome,
    ":email"    => $email,
    ":telefone  => $telefone
]);
```

Método de Vinculação de Valores: `bindValue()` Vs. `bindParam()`

Ao associar parâmetros a uma consulta `prepare`, pode-se utilizar dois métodos com comportamentos distintos. o `bindValue()` ou o `bindParam()`, o primeiro vincula um valor fixo no momento da chamada, enquanto o segundo vincula uma variável por referência e só avalia o valor real quando a consultal é executada.

Exemplo `bindValue()`: Associa o valor exato da variável naquele momento, é mais comum e seguro para 95% dos casos de uso.

```
$id = 10;
$stmt->bindValue(":id", $id, PDO::PARAM_INT);
$id = 20; //não altera o valor que já foi passado no bindValue!
$stmt->execute(); //executa com id = 10
```

Exemplo `bindParam()`: Associa a variável como uma referência de memória (`&`). O valor é lido no momento exato da chamada `execute`. Usar apenas em loops ou situações específicas que precisa alterar o valor repetidamente.

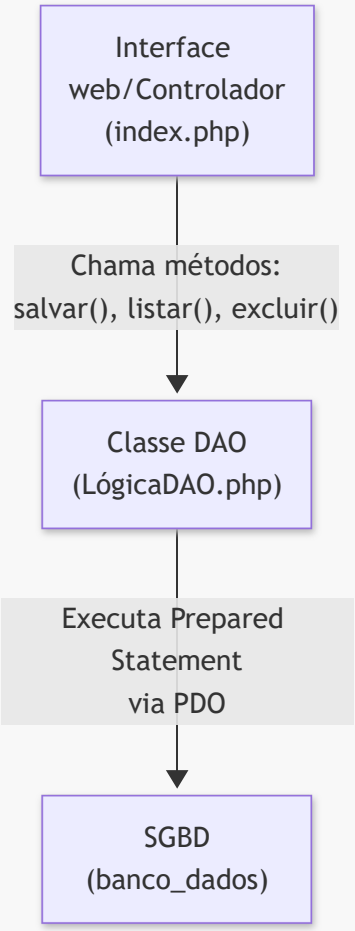
```
$id = 10;
$stmt->bindParam(":id", $id, PDO::PARAM_INT);
$id = 20; //altera o valor de referência
$stmt->execute(); //executa com id = 20
```

> obs: Tipagem Explícita com Constantes do PDO: Para Garantir que o PostgreSQL interprete corretamente o tipo de dado, inform-se sempre a constante correspondente:
>
> - `PDO::PARAM_INT`: Para chaves primárias, quantidades e inteiros.
> - `PDO::PARAM_STR`: Para textos, strings, datas e números decimais (`NUMERIC`).
> - `PDO::PARAM_BOOL`: Para valores booleanos (`true`/`false`).
> - `PDO::PARAM_NULL`: Para passar valores nulos explícitos.

O Padrão de Arquitetura DAO(DATA ACCESS OBJECT)

Em aplicações profissionais, comando SQL nunca devem ser escritos diretamente dentro de arquivos de interface visual (como páginas HTML ou controladoras de tela)

Para separar a **lógica de apresentação** da **lógica de acesso a dados**, usa-se o padrão de projetos **DAO(DATA ACCESS OBJECT)**:



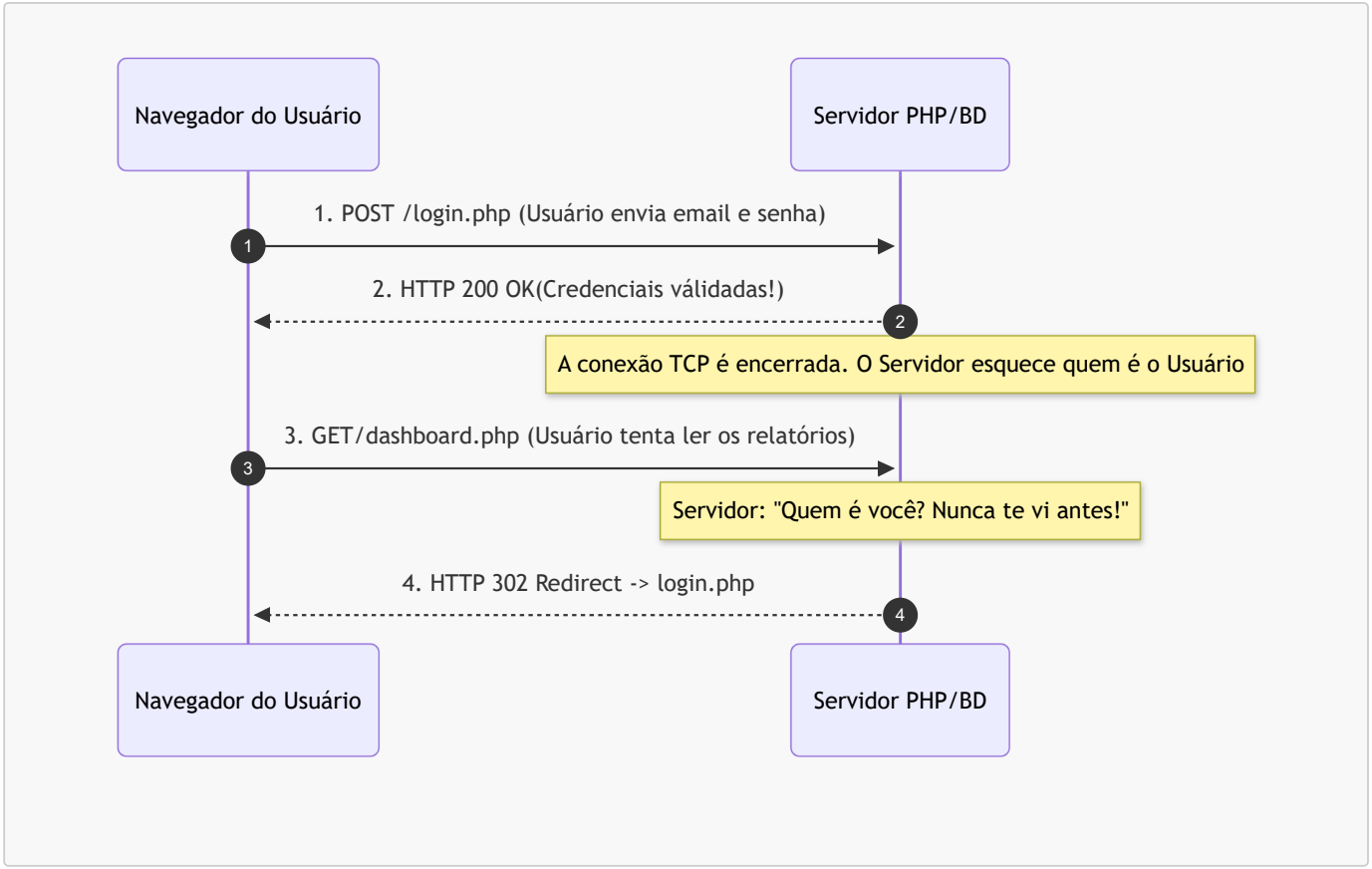
Semana 10 - Sessões, Cookies e Autenticação Segura

Tema: Gestão de Estado com HTTP, Ciclo de Vida da Sessão(`$_SESSION`), Cookies, Hashing de Criptografia, Middleware de Autenticação

O Protocolo HTTP: Como ele lembra de você?

O Protocolo HTTP é totalmente **STATELESS (sem armazenamento de estado)**. Isso Significa que o servidor web trata cada requisição HTTP de forma isolada, como se fosse a primeira vez que ele estivesse interagindo com o usuário do sistema.

Situação sem Armazenamento de Cookie e SESSION



Para resolver esse problema sem obrigar o usuário a digitar o logine senha novamente a cada clique fazemos o seguinte:

1. Ao fazer o login bem-sucedido, o servidor entrega ao cliente uma **comanda/pulseira VIP numerada** (um identificador de sessão unica e aleatória)
2. O navegador armazena essa comanda/pulseira em um **Cookie**
3. A cada nova página solicitada, o navegador apresenta automatiocamnet essa pulseira no cabeçalho da requisição
4. O PHP l^o número da comanda/pulseira, localiza os dados do usuário aramazenado na memoria do servidor(`$_SESSION`) e reconhece quem é ele!

Situação com Armazenamento de Cookie e SESSION



Como funciona o Cookie HTTP

Um **Cookie** é um pequeno arquivo de texto(geralmente limitado a 4KB) gravado pelo navegador a pedido do servidor, através do cabeçalho de resposta `Set-Cookie`

Cookie podem vazar Informações

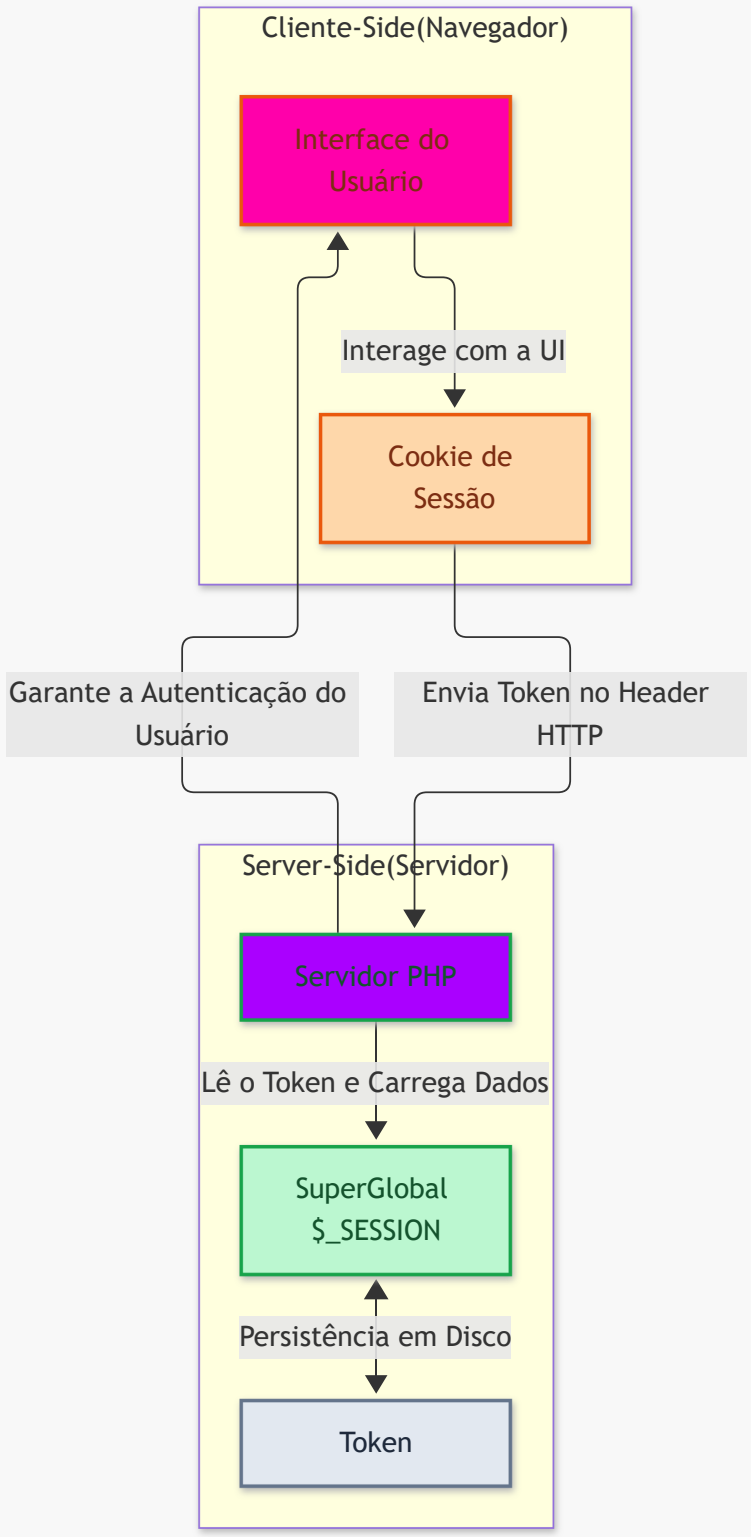
Sim, se um cookie de sessão for configurado com parâmetros desprotegidos, qualquer script malicioso injetado na página(XSS) pode roubar a sessão do usuário logado.

Para evitar problemas de Segurança em Cookie: usando Flags de segurança.

Flag de Segurança	O que ela faz?	Por que é indispensável?
`HttpOnly`	Impede que scripts JavaScript executem leitura no cookie via `document.cookie`.	**Neutraliza o roubo de sessão via XSS!** Mesmo que haja uma falha de injeção de script na tela, o navegador proíbe o JavaScript de ler o identificador de sessão.
`SameSite=Lax`	Restringe o envio do cookie apenas a navegações originadas do próprio domínio.	**Protege contra ataques CSRF** (*Cross-Site Request Forgery*), impedindo que links externos forjem ações logadas.
`Secure`	Garante que o cookie só seja transmitido em conexões criptografadas com **HTTPS**.	Impede que bisbilhoteiros em redes Wi-Fi públicas interceptem a sessão em trânsito (*Man-in-the-Middle*). *(Em ambiente local de desenvolvimento sem SSL, pode ser desativado temporariamente).*

O Ciclo de Vida da SESSÃO em PHP: `$_SESSION` no servidor

Enquanto os cookies residem no Navegador, os dados da sessão ficam armazenados com **privacidade total dentro do servidor** (em um arquivo temporário na memória)



Sintaxe Essencial do PHP para Sessão

```
//configurar as sessão com funções nativas do PHP(ler documentação)

//1. Configurar as flags dos cookies Antes de abrir a sessão
session_set_cookie_params([
    "lifetime"      => 0,       // Cookie expira ao fechar o navegador
    "path"          => "/"      // Válido para todas as páginas do dominio
    "httponly"      => true     // Proteção contra XSS
    "samesite"      => "Lax"    // Proteção contra CSRF
```

```
]);

// 2. Iniciar ou restaurar a session ativa
session_start();

// 3. Gravar os dados da sessão (após validar o login)
//usar seperglobais($_SESSION) para isso
$_SESSION["usuario_id"] = 10;
$_SESSION["usuario_perfil"] = "ADMIN";

// 4. Permitir a leitura dos dados em qualquer página do meu site
$idLogado = $_SESSION["usuario_id"] ?? null; // se o usuario não tiver feito login a session é
null
```

Criptografia de Senhas

No passado, desenvolvedores armazenavam senhas utilizando funções matemáticas de integridade como `md5($senha)` ou `sha1($senha)`.

> O MD5 foi quebrado em 2004 e SHA1 foi quebrado em 2017.

A Solucção Moderna é Criptografia com uso de Hash

O padrão de criptografia utilizado são o `Argon2id`, ou o `Bcrypt`

Caracterisiticas da Criptografia de Hash

1. **Unidirecional(One-way):** É matematicamente impossível descriptografar o hash para recuperar a senha original, já que o hash muda cada milisegundo.
2. **Salt Automático e Aleatório**: A cada execução, a função gera um vetor de `salt` único de 16bytes. Mesmo que 2 pessoas tenham a mesma senha, os hashes gerados serão completamentes diferentes!.
3. **Custos Ajustáveis**: É possível calibrar quanto de memória o algoritmo levará para calcular o hash

Sintaxe Profissional de Cryptografia Moderna

```
$senhaDigitada = *******
//
$hash = password_hash($senhaDigitada, PASSWORD_ARGON2ID);
//saída:  $argon2id$v=19#m65536,t=4,p=1$jkhdsfhl...(comprimento ~96 a 128 chars)
```

O processo de Descriptografia

```
$senhaInformada = $_POST["senha"];
$hashDoBanco    = $usuario["hash"];

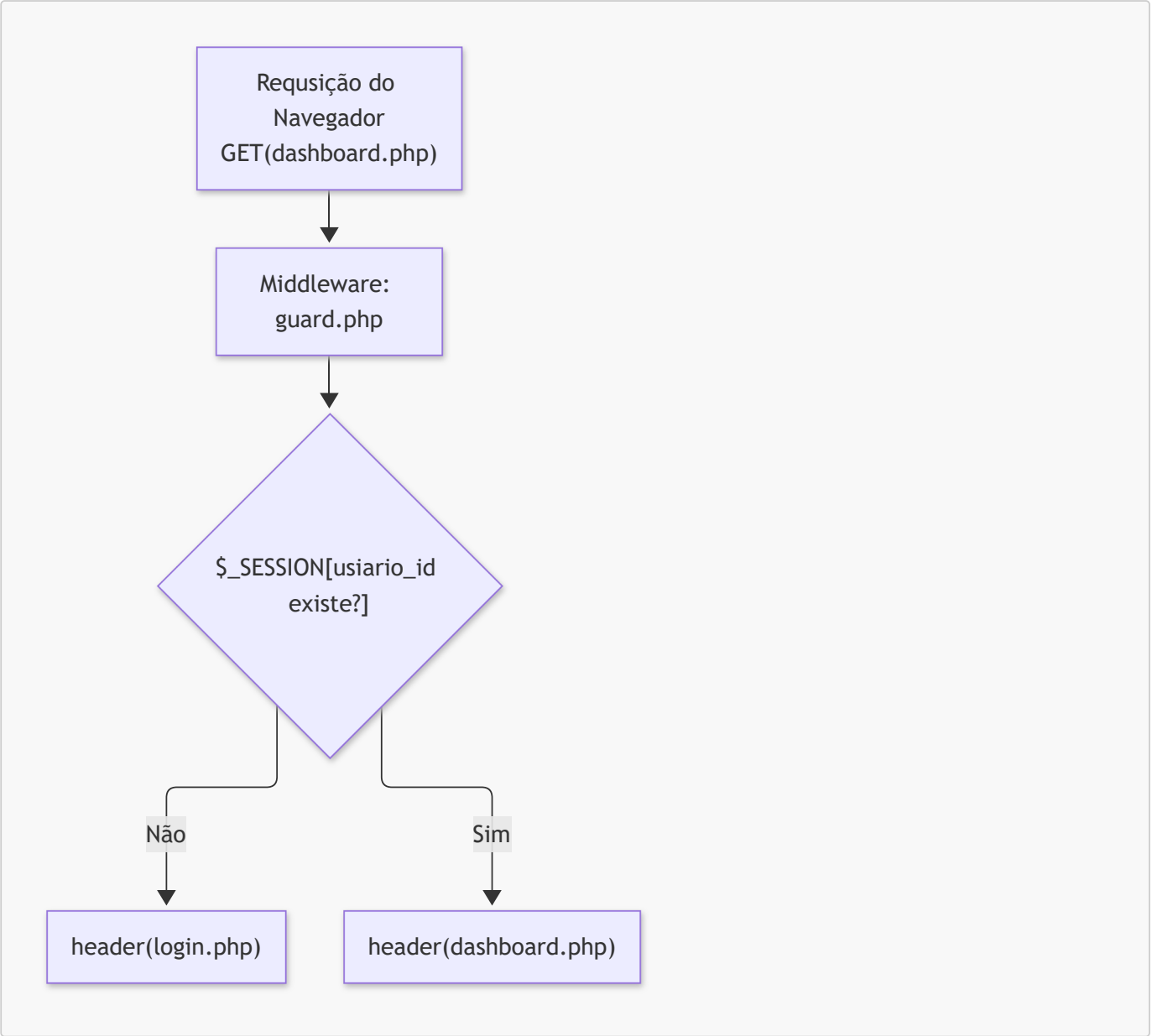
//o processo de verificação
if(password_verify($senhaInformada, $hashDoBanco)){
    //Senha Correta
} else{
    //Senha Incorreta
}
```

> [Aviso|] Nunca tente fazer (password_hash($senha) === $hashDoBanco) Como o `password_hash` gera um salt aleatório a cada milisegundo, a comparação por igualdade **sempre será falso** A Verifiação deve ser feita

> **exclusivamente** com a função `password_verify()`

Arquitetura de Autenticação: Middleware (guard)

Para impedir que visitantes não logados acessem páginas privadas (como dashboard.php ou relatorios.php), criamos interceptadores chamados **Guards (Middlewares de Proteção)**.



Como usar o guard.php:

Em todas as páginas restritas do sistema a **primeira linha de código** após o `declare(strict_types=1)` será:

```
required_once __DIR__ . "/src/guard.php";
```

Se a pessoa não estiver autenticada ou não tiver o perfil de acesso, ela é expulsa antes mesmo que qualuqer byte de HTML seja renderizado.

Semana 11 - Programação Orientada a Objetos em PHP.

Tema: Paradigma de POO, os 4 Pilares da Orientação a Objetos (Abstração, Encapsulamento, Herança e Polimorfismo), Classes e Instâncias, Construtores, Modificadores de Acesso (`private`, `protected` e `public`) e Modelagem de Dados.

Do Procedural à Orientação a Objetos

Na Programação Procedural manipulação de variáveis, laços de repetição e funções são interpretados de forma linear, ou seja, todo o código é escrito em um monobloco (`código de um único arquivo de programação`). Essa abordagem resolve problemas lineares com maestria, mas colapsa rapidamente quando sistemas crescem em escala, complexidade e criticidade de regra de negócio.

Na Programação Procedural:

1. **Os dados estão exposto**: Qualquer função pode alterar qualquer campo a qualquer momento.
2. **As regras de validação ficam dispersos**: Cada tela precisa checar uma função, gerando dezenas de `if/else`duplicados espalhados pelo código. Se uma regra mudar, o desenvolvedor precisa lembrar de atualizar todos os arquivos do sistema.
3. **Não há garantia de invariantes**: É impossível garantir que um dado sempre permaneça em estado consistente.

O que é Paradigma de Programação Orientado a Objetos (POO)?

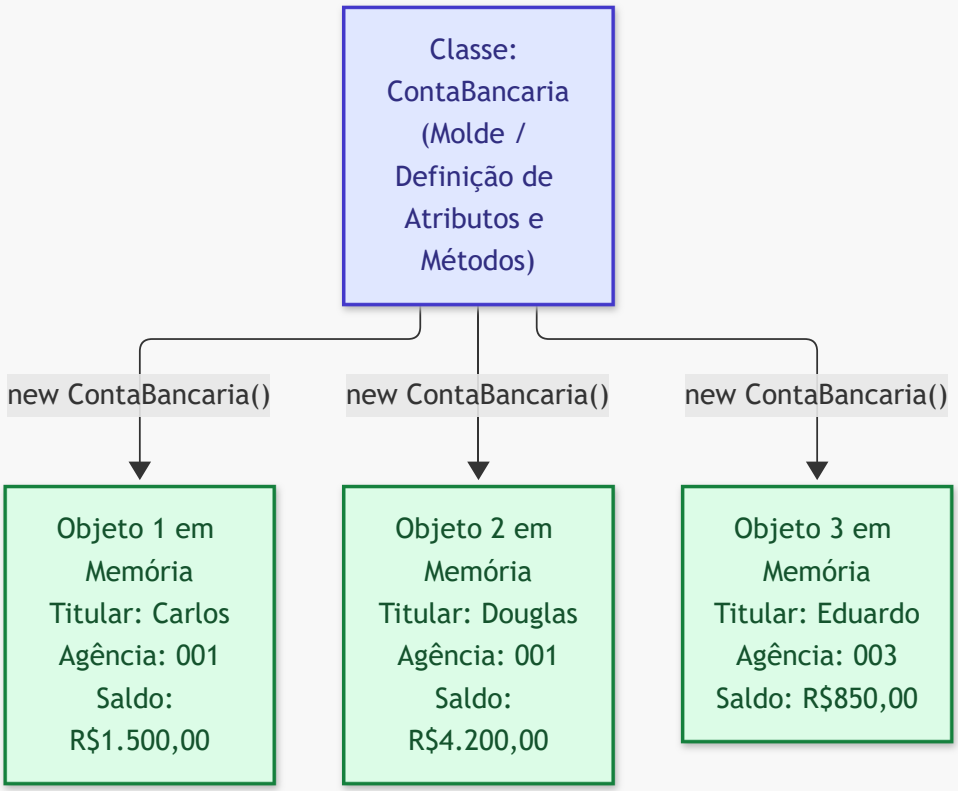
Na **POO**, em vez de separar funções de um lado e dados soltos de outro, nós unimos *dados(atributos)* e as *funções que manipula esses dados(métodos)* dentro de uma única estrutura chamada **Objeto**

Quadro Comparativo Procedural vs POO:

Aspectos	Paradigma Procedural	Praradigma Orientado a Objetos(POO)
Unidade Fundamental	Procedimento e Funções soltas	Classes e Objetos
Proteção de Dados	Nenhuma (dados globais ou em arrays abertos)	Alta (Encapsulamento)
Reuso de Códigos	Inclusão de Script (`required_once`) e cópia	Herança(`extends`), Composição e Interfaces
Tratamento de Variaçoes	Blocos gigantes de `switch` e `if/else`	Polimorfismo e Classes Abstratas
Manutenibilidade	Complexa à medida que a base de código cresce	Alta, modular, testável e pronta para arquitetura corporativas (MVC, DDD, Clean Architecture)

Anatomia da Classe e Objeto

- **Classe**: é molde, a planta baixa, o projeto de engenharia. Ela descreve quais atributos(dados) e comportamentos(métodos) as entidades daquele tipo possuirão. Ela reside no código-fonte e não ocupa espaço dinâmico de dados na memória.
- **Objeto**: é o produto final construtido a partir do molde, uma instância real, palpável e viva alocada na memória do servidor durante o ciclo de execução.



O Ciclo de Instanciamento de Objetos

Ao instanciar um objeto com a instrução `new`, o PHP invoca o método constructor especial `__construct()`. E usamos a palavra-chave `$this`para fazer referência direta e exclusiva ao **atributo** que está sendo executado pelo código naquele momento.

```php
class ContaBancaria {
    // Declaração tradicional de atributos tipados
    private string $titular;
    private int $agencia;
    private float $saldo;

    // Método Construtor
    public function __construct(string $titular, int $agencia, float $saldo){
        $this->titular = titular; //this->titualr refere-se ao atributo da classe
        $this->agencia = agencia; //this->titualr refere-se ao atributo da classe
        $this->saldo = saldo; //this->titualr refere-se ao atributo da classe
    }

    // Método de Acesso
    public function getSaldo(): float{
        return $this->saldo;
    }
} //fecha a classe

//Instanciando dois objetos completamentes independentes
$conta1 = new ContaBancaria("Carlos", 001, 100.00);
$conta2 = new ContaBancaria("Douglas", 002, 500.00);
```

```php
echo $conta1->getSaldo(); // Exibir 100.00
echo $conta2->getSaldo(); // Exibir 500.00
```

> [!OBD:] **Alocação de Objetos na Memória** No PHP, variáveis que armazenam objetos, usam um **identificador/referência** para o endereço na memória. Quando você passa um objeto para uma função ou método, está passando a referência na memória sobre aquele objeto, o que torna o POO extremamente performático.

Os 4 Pilares da Orientação a Objetos



1. **Pilar 1: Abstração Definição**: Capacidade de filtrar detalhes irrelevantes e focar estritamente nas características e comportamentos essenciais da entidade para o contexto do software, que está sendo construido. *Exemplo:* Para um sistema bancário, o que importa em um cliente é seu nome, CPF e renda. Sua altura ou cor favorita são abstrações e podem ser descartados.

2. **Pilar 2: Encapsulamento Definição**: Bildagem dos estads internos do objeto. Os atributos devem ser protegisdos conta acessos ou modificações diretas pelo mundo exterior. Alterações no estado só ocorrem através de métodos públicos que validam regras de negócio e mantêm invariantes contábeis.

3. **Pilar 3: Herança Definição**: Mecanismo pelo qual uma classe filha (subclasse) herda atributos e comportamentos de uma classe ancestral (superclasse), promovendo o reuso de códigos e estabelecendo uma relação do tipo **É UM** *Exemplo*: Um Cachorro(subclasse) é um Mamífero(superClasse)=> porque ele herda caracteristicas da classe dos Mamíferos.

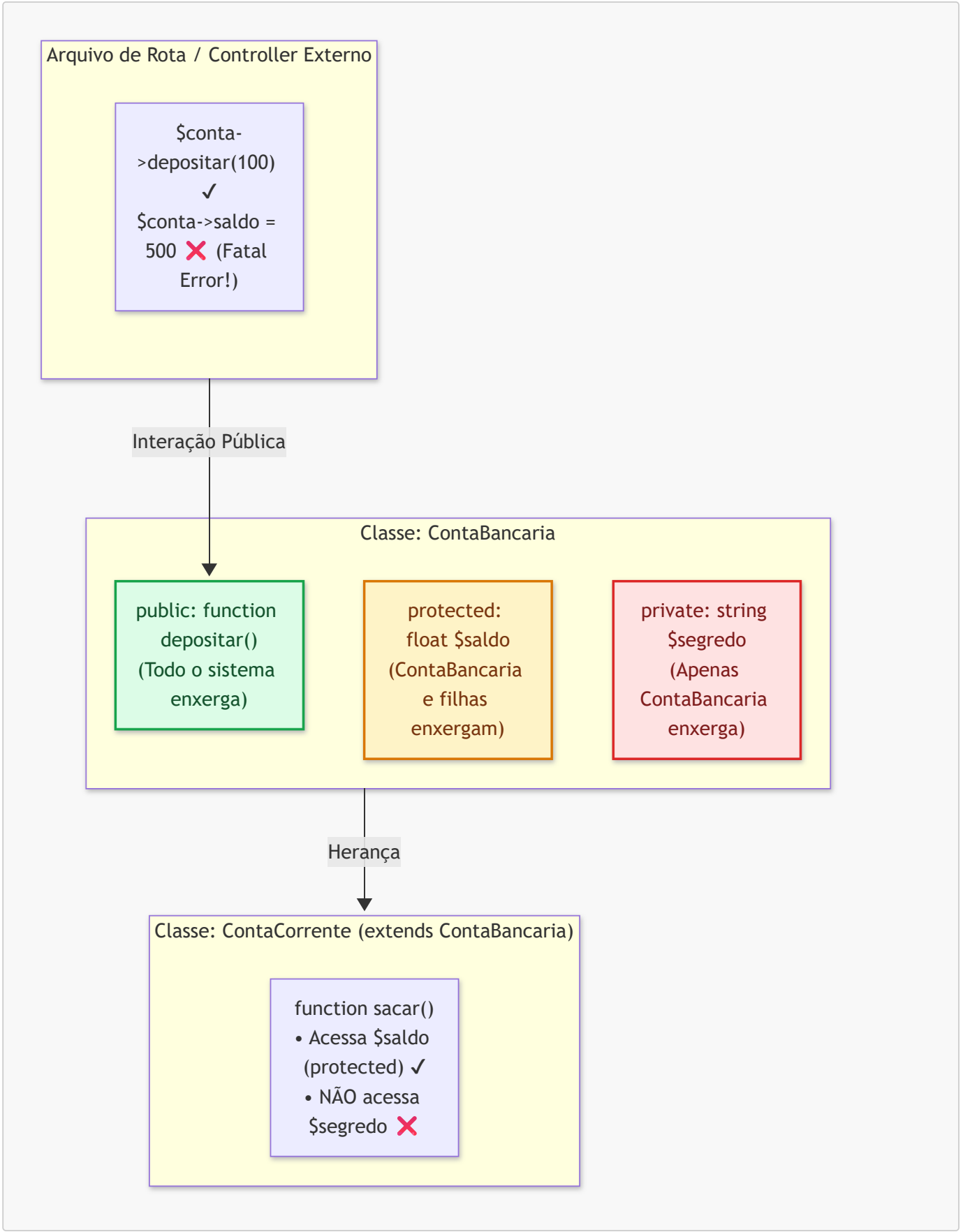
4. **Pilar 4: Polimorfismo Definição**: Princípio que permite que classe filhas diferentes respondam a mesma chamada de método, porém, com mimplementações especializads e comportamentos distintos. O chamador não precisa saber qual subclasse exata está executando, apenas confia que o método cumpra o contrato.

Encapsulamento

Os Modificadores de Visibilidade:

Modificador	Onde pode ser acessado?	Recomendação de Uso
`private`	**Apenas** dentro da própria classe que o declarou. Subclasses e scripts externos não conseguem ler nem alterar.	**Padrão ouro para atributos.** Garante proteção absoluta de estado e impede efeitos colaterais.
`protected`	Dentro da própria classe e por quaisquer **subclasses que a herdem (`extends`)**. O mundo exterior continua bloqueado.	Usado quando subclasses precisam ler ou ajustar diretamente um valor compartilhado (ex.: `$saldo` em classes filhas de conta).
`public`	Acessível livremente por **qualquer parte do sistema**, dentro ou fora da classe.	Usado para os métodos que compõem a **interface pública** do objeto (ações como `sacar()`, `depositar()`, `transferir()`).

Gráfico Representativo dos modificadores de acesso em Heranças



Usando Getters and Setters

Ao Declarar atributos privados ou protegidos (`private` ou `protected`), se você deseja manipular esses atributos é importante criar os **Modificadores de Acesso** atraavés dos `getters and setters`

Exemplo: Ao criar um atributos `private float $saldo`, para modificar os valores do saldo ou ler o saldo é importante criar os métoso públicos.

- `public function getSaldo()` : Permite consultar o valor do saldo
- `public function setSaldo()` : Permite alterar o valor do saldo

> [!Cuidado] **Não destrua seu encapsulamento**: para atributos que não serão modificados, como CPF, por exemplo, não há necessidade de criar um `setCPF()`, o CPF de uma pessoa não será modificado alo longo da vida.

Herança, Classes Abstratas e Polimorfismo

Classe Abstrata: Uma Classe Abstrata é uma superclasse conceitual que **não pode ser instanciada diretamente com** `new`. Ela serve como matriz para que subclasses concretas sejam derivadas.

```
abstract class ContaBancaria {
    //superclasse -> passa informações para as subclasse (herança)
    protected float $saldo;
    //construtor
    public function __construct(float $saldoInicial){
        $this->saldo = $sandoInicial;
    }
    //métodos de acesso (getters and setters)
    public fucntion getSaldo(): float {
        return $this->saldo;
    }
    //método abstrato: Obriga a subclasse a criarem, porem não vai fornecer a implemetação do
método(polimorfismo)
    abstract public function sacar(float $valor):void;
}

//Se um objeto da classe ContaBancaria tentar ser instanciado, vai gerar um erro na compilação
$conta =  new ContaBancaria(100); //Error:Cannot instantiate abstract class ContaBancaria
//Não pode instanciar OBJ da Classe Abstrata
```

Herança: Usar o mecanismo de criação de subclasse a partir da superclasse (extends)

```
class ContaCorrente extends ContaBancaria {
    //não precisa redeclarar os atributos e métodos já criados na superclasse
    //a unica coisa que preciso implementar é o método sacar
    public function sacar(float $valor): void {
        //Implemetar a lógica de negocio para ContaCorrente
        $this->saldo -= $valor + 2.50; //Taxa de Saque de 2.50
    }
}

class ContaPoupanca extends ContaBancaria {
    //outro método para sacar
    public fucntion sacar(float $valor) : void {
        //implemetar a lógica de negócio para ContaPoupanca
        if($valor <= $this->saldo>){
            $this->saldo -= $valor; // não tem tx
        }else{
            echo "Não pode Sacar esse valor";
        }
    }
}

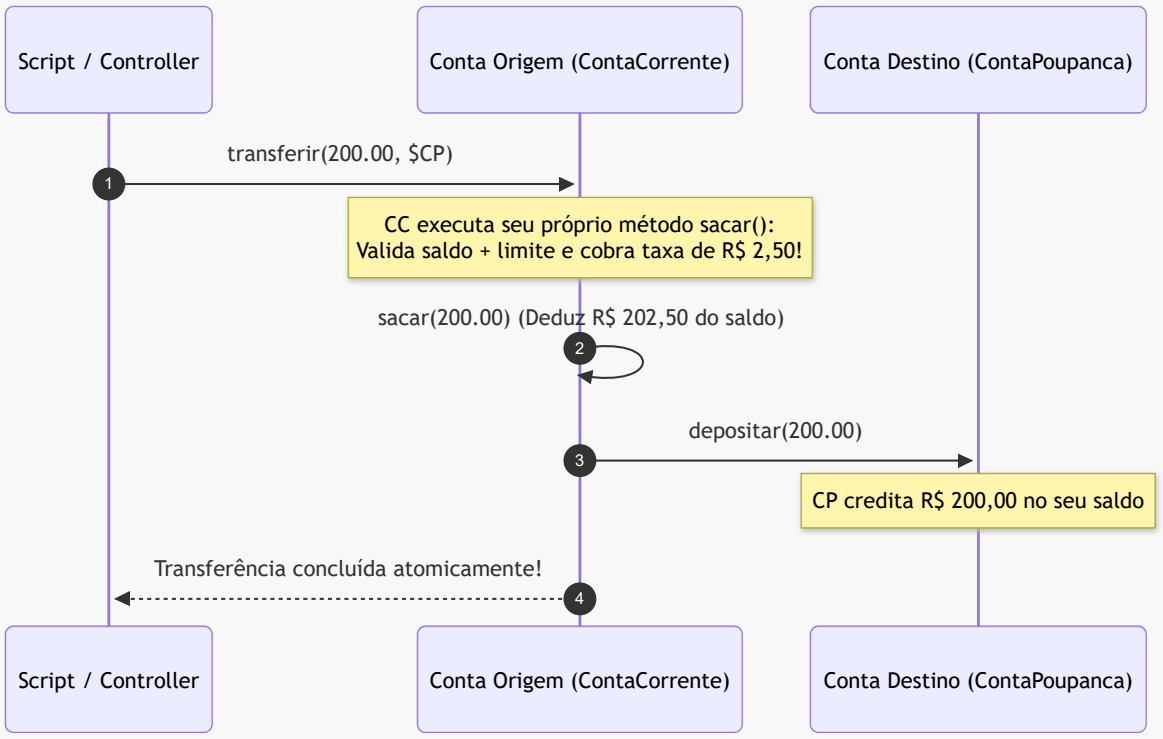
$contaCorrente = new ContaCorrente(1000.00);
$contaPoupanca = new ContaPoupanca(500.00);
```

```php
$contaCorrente->sacar(100.00); //Deduz R$ 102,50
$contaPoupanca->sacar(100.00); //Deduz R$ 100.00

$contaCorrente->getSaldo(); // Retorna 897,50
$contaPoupnaca->getSaldo(); // Retorna 400,00
```

> Observação: ambas as classe implementam o método `sacar()` de maneira diferentes, respeitando as regras de negócio específicas de cada tipo de conta.

Polimorfismo: O método `sacar()` na classe ContaBancaria demonstra o polimorfismo em ação.



Técnicas e Tratamento de Exceções

Constructor Property Promotion (Declaração no Construtor)

No PHP 8.X Podemos utilizar recursos modernos que reduzem o código repetitivo e elevam a segurança. O uso do `Property Promotion` no construtor é um exemplo, não precisamos declarar os atributos e depois fazer o construtor, basta declarar os atributos diretamente dentro do construtor.

```php
class TitularProperty {
    public function __construct(
        private string $nome,
        private string $cpf
    ){}
}

//isso é igual

class TitularSemProperty {
    private string $nome;
    private string $cpf;
```

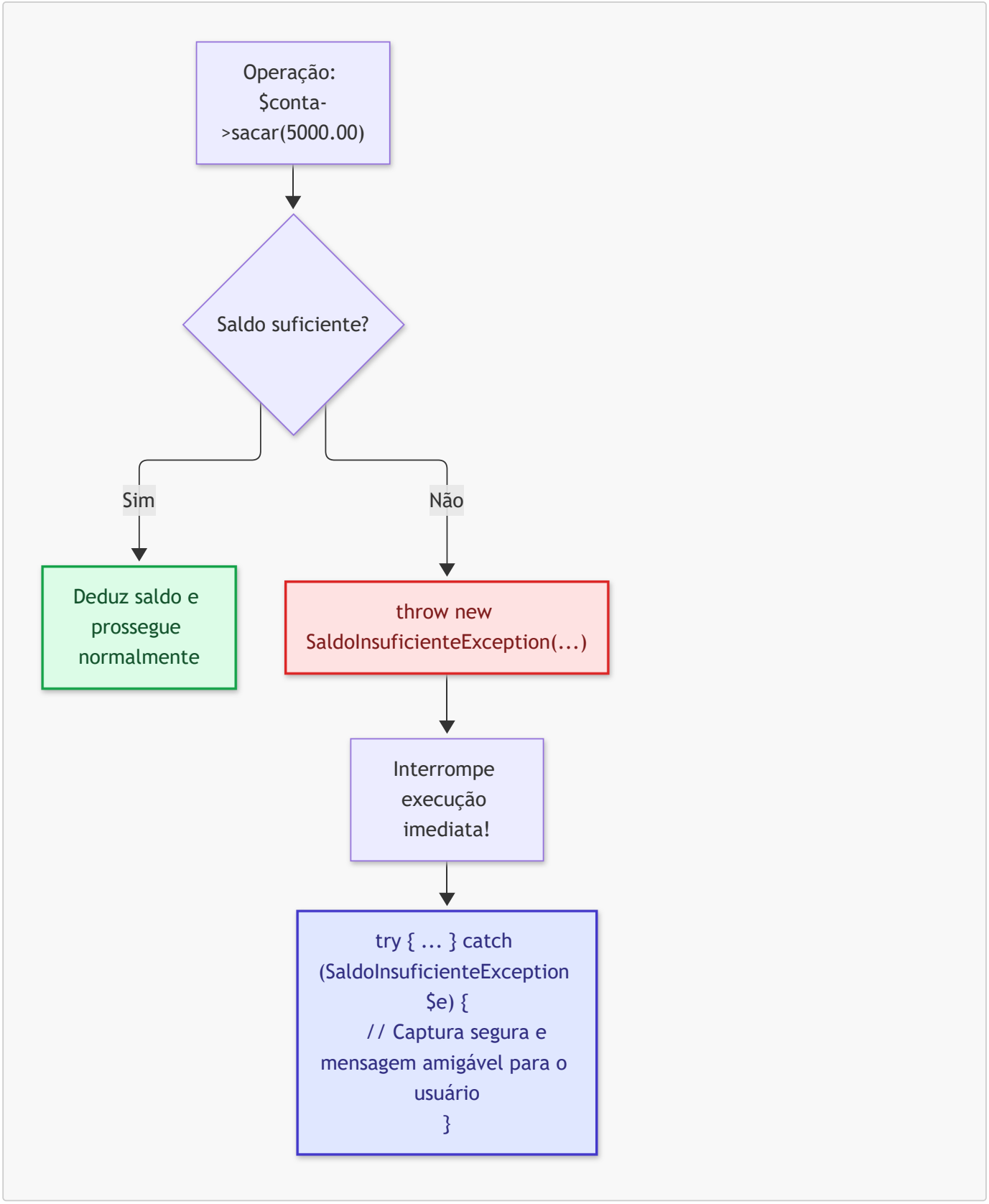
```
    public function __construct(string $nome, string $cpf){
        $this->nome = $nome;
        $this->cpf = $cpf
    }
}
```

Exceções de Domínio: Tratamento de Erros do Sistema

Um dos maiores erros cometidos em sistemas procedurais é o retorno mágico de valores como `-1`, `false` ou `null` quando uma regra de negócio é violada.

Para Corrigir esses Erros Mágicos devemos usar `throw Exception`

Se o método retornar `false` quando ocorrer um erros, o desenvolvedor não fez o tratamento de erros corretamente. Exemplo: Se o `sacar()` retornar `false`quando não houver saldo, o sistema continuará executando como se o dinheiro tivesse saído causando furos de caixa no sistema. Já quando usamos uma **Exceção de Domínio**, a execução é interrompida e feito um tratamento usando um bloco `try/catch`.



> [!OBS] `try/catch` é o mecanismo de captura de exceções do PHP. Ele permite que você trate erros de forma inteligente, mantendo a integridade do sistema e fornecendo feedback claro ao usuário.

Semana 12 - Arquitetura MVC Artesanal

Tema: Pradrão Arquitetural MVC(Model-View-Controller) em BackEnd. Gestão de Dependências Composer, Especificações de Autoloading PSR-4, Resolução Automática de NamesSpace, Padrão DrontController, Roteador HTTP Artesanal e Injeção de Dados em Views.

O fim do Código Espaguete

Quando encontramos um padrão de código desastroso e caótico, sem arquiteturas modernas de Desenvolvimento, usamos o termo `Código Espaguete`.

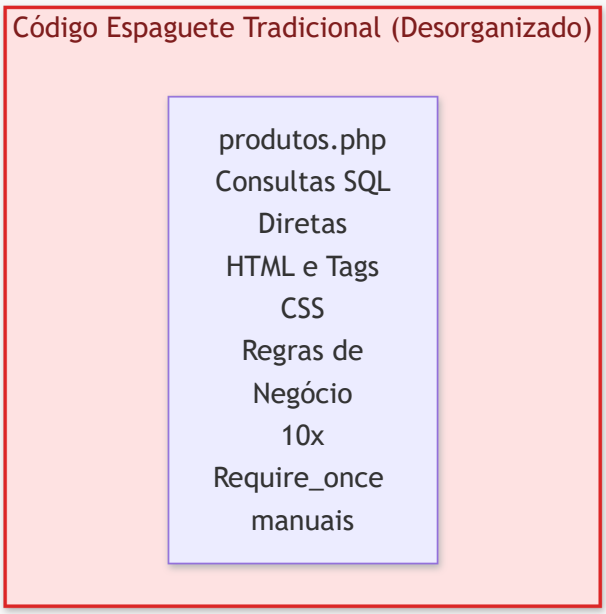
Exemplo de Código Espaguete:

```
<!-- ARQUIVO LEGADO: produtos.php (O PESADELO DO CÓDIGO ESPAGUETE) -->
<?php
// 1. Múltiplos includes manuais frágeis
require_once __DIR__ . '/../config/conexao.php';
require_once __DIR__ . '/../classes/Produto.php';
require_once __DIR__ . '/../classes/Sessao.php';
require_once __DIR__ . '/../classes/Auditoria.php';

// 2. Mistura de lógica de banco e processamento HTTP
$pdo = Conexao::getInstancia();
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $nome = $_POST['nome'];
    $stmt = $pdo->prepare("INSERT INTO produtos (nome) VALUES (?)");
    $stmt->execute([$nome]);
}
$produtos = $pdo->query("SELECT * FROM produtos")->fetchAll();
?>
<!-- 3. HTML misturado com PHP procedural na mesma folha -->
<html>
<body>
    <h1>Catálogo</h1>
    <?php foreach($produtos as $p): ?>
        <p><?= $p['nome'] ?></p>
    <?php endforeach; ?>
</body>
</html>
```

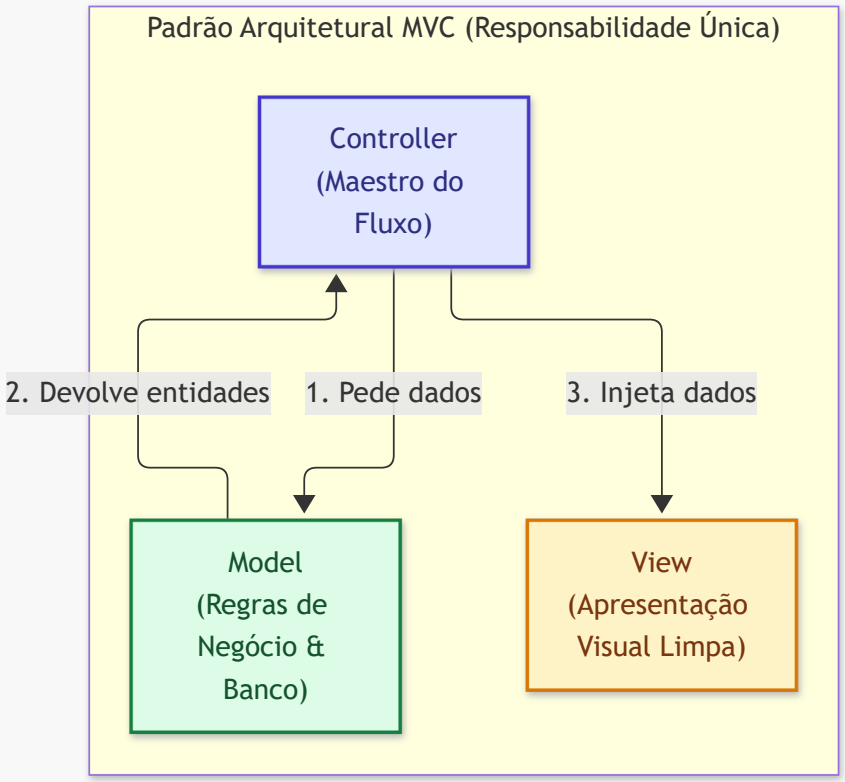
Os Problemas Críticos do Código Espaguete

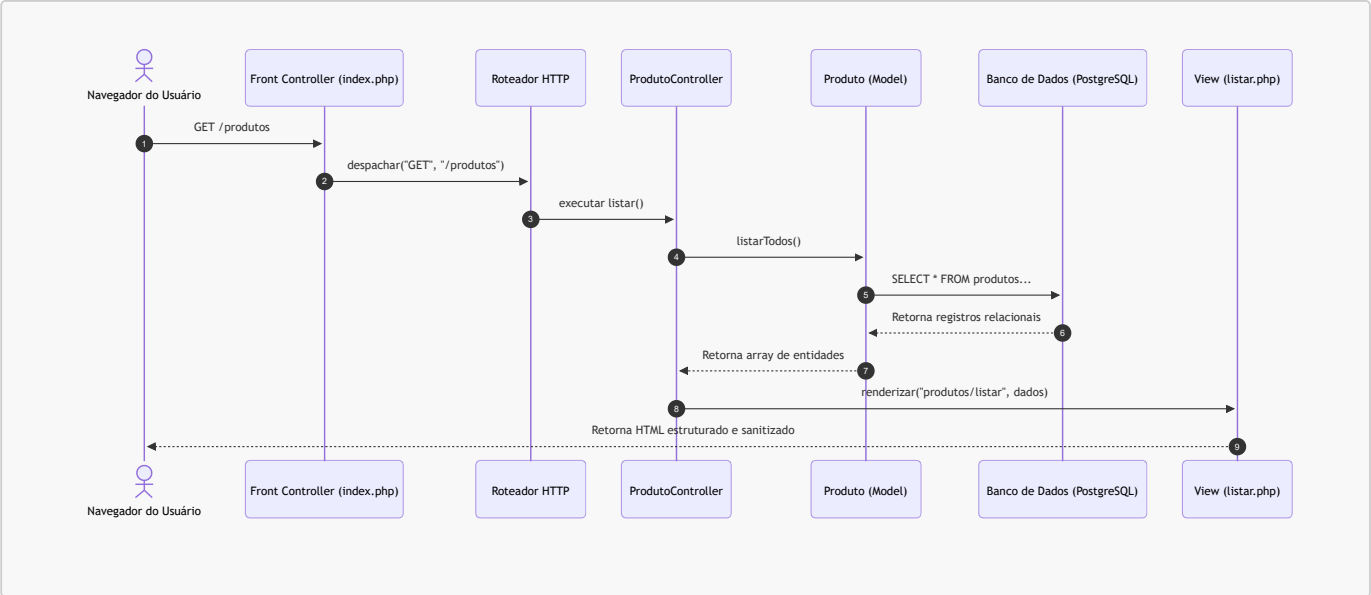
1. *Acoplamento Máximo*: Se você mudar o layout visual, corre o risco de quebrar uma query SQL. Se alterar uma coluna do banco, precisa reescrever o arquivo vicual.
2. *Fragicilidade de Inclusões (`required_once`):* Se um arquivo for movido de pasta, todas as chamadas `required_once`quebram com erros fatais de arquivos não encontrados.
3. *Exposição de Segurança*: Se os arquivos PHP estiverem na raiz acessível do servidor web, qualquer usuário malicioso pode tentar acessar scripts internos digitando `http://site.com/config/conexao.php`.



****A Triade Model-View-Controller (MVC) no BackEnd**

O Padrão **MVC (Model-View-Controller)** é a arquitetura de software corporativa mais consagrada da história da web. Seu Objetivo é separar as preocupações do sistema em **3 camadas bem definidas**:





1. **Model (Camada de Modelo de Negócio e Dados)**

- **O que Faz**: Representa as entidade do mundo real e gerencia a lógica de dados, persistência e regras de negócio
- **O que sabe**: Sabe como conectar ao banco de dados, executar Prepared Statement, validar se um preço é positivo (RN) e calcular totais de estoque(RN)
- **O que NUNCA faz**: Nunca emite comando `echo`, nunca gera tag HTML e nunca lê dados da superglobal `$_POST` ou `$_GET`.

2. **View (Camada de Visualização e Apresentação)**

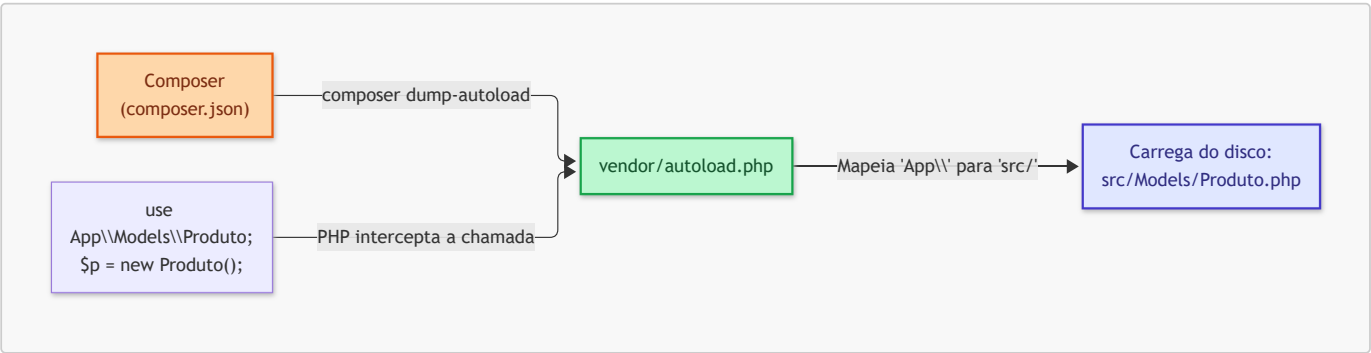
- **O que Faz**: Renderiza a interface gráfica do usuário(geralmente HTML, CSS, JSON).
- **O que sabe**: Recebe variáveis simples enviadas pelo Controller e as formata visualmente, aplicando sanitizção ativa contra XSS(`htmlspecialchars`)
- **O que NUNCA faz**: Nunca executa consultas SQL, nunca faz regras de negócios complexa e nunca redireciona requisições via `header()`

1. **Controller (Camada de Controle e Orquestração)**

- **O que Faz**: Atua como o maestro da aplicação. REcebe a requisição do usuário, valida as entradasm aciona a camada Model para obter ou persistir informações e seleciona a View correspondente para renderizar a resposta
- **O que sabe**: Sabe qual Model chamar e qual View devolver.
- **O que NUNCA faz**: Não deve conter regras contábeis complexas (elas pertencem ao Model) e não deve escrever HTML diretamente.

O Gerenciador Composer e o AutoLoad PSR-4

Imagine um projeto coroporativo moderno com mais de 300 classes. ter que escrever 300 linhas de `required_ondeem` cada arquivo ternaria o desenvolvimento inviável. É aqui que entra o **Composer** e a especificação **PSR-4**



O que é o Composer?

O **Composer** é o gerenciador de dependências oficial da linguagem PHP. Ele desempenha duas funções vitais:

1. Baixar bibliotecas de terceiros a partir do repositório central Packagist (como bibliotecas de envio de e-mails, geradores de PDF e frameworks).
2. **Gerar o Autoloader Automático:** O Composer lê as classes criadas por você no seu projeto e as disponibiliza automaticamente no código sem a necessidade de nenhum `require`.

As Especificações do PSR-4 (PHP Stadart Recommendation 4):

A especificação internacional **PSR-4** dita que:

> *O namespace de uma classe deve espelhar exatamente a estrutura de diretórios no disco físico onde o arquivo está gravado.*

Exemplo prático:

- Se declaramos o namespace: `namespace App\Controllers; class ProdutoController {}`
- E configuramos no Composer que o prefixo `"App\\"` aponta para a pasta `"src/"`.
- O PHP localizará fisicamente o arquivo em: `src/Controllers/ProdutoController.php`.

Como Configurar o composer.json

```
{
    "name": "senai/supply-mvc",
    "description": "Sistema de Gestão de Insumos Industriais",
    "type": "project",
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        }
    }
}
```

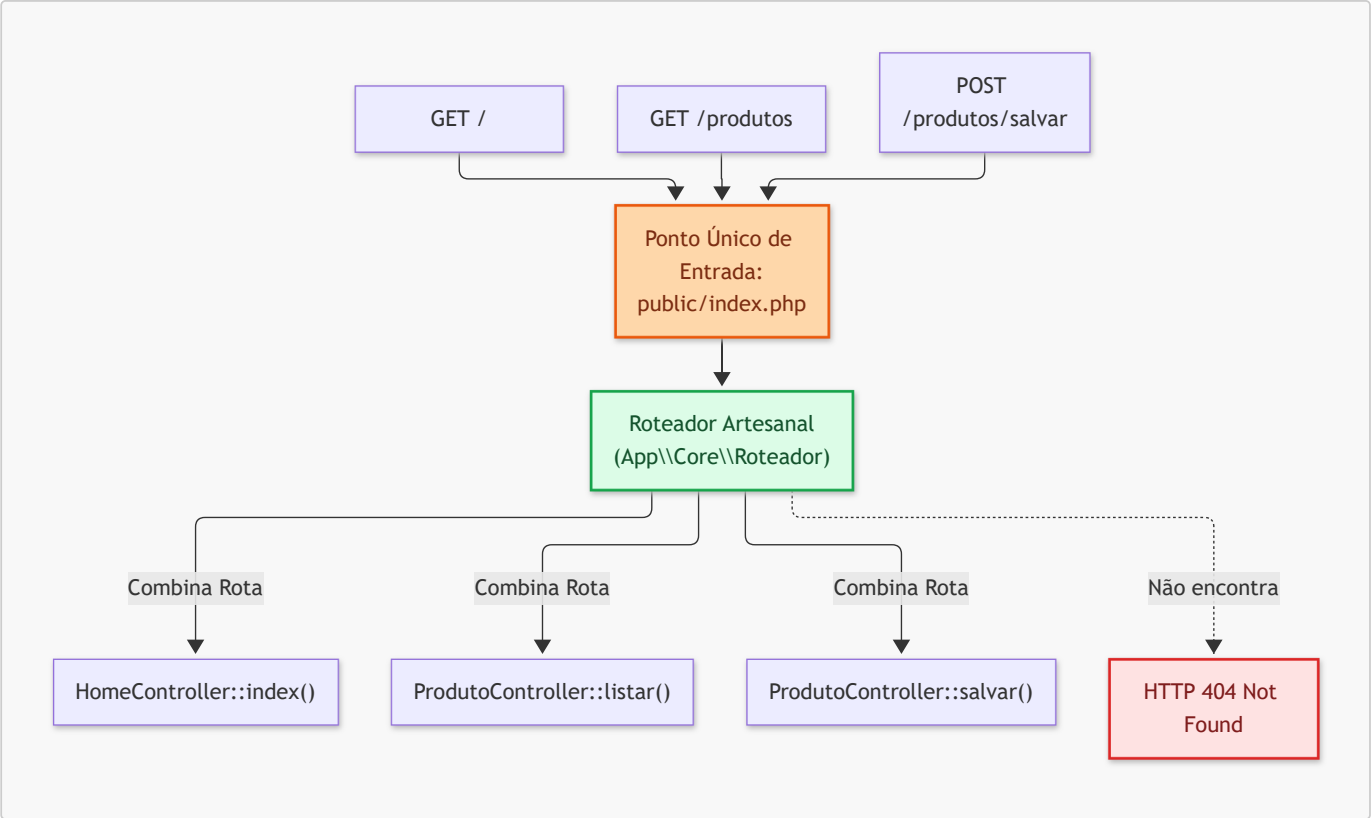
> [!IMPORTANT] **Comando Mandatório de Regeneração:**
> Sempre que você criar o arquivo `composer.json` ou alterar a estrutura de namespaces, execute no terminal do VS Code:
> `composer dump-autoload -o`
> O parâmetro `-o` gera o mapa de classes otimizado para máxima performance.

O Padrão Front Controller e Roteamento HTTP

Em aplicações PHP procedurais antigas, a URL do navegador correspondia exatamente a um arquivo gravado no disco:

- `http://site.com/contato.php` ➔ Carrega o arquivo `contato.php`.
- `http://site.com/admin/relatorio.php` ➔ Carrega o arquivo `admin/relatorio.php`.

Em arquiteturas corporativas modernas (incluindo **Laravel**, **Symfony** e **Spring Boot**), essa correspondência direta não existe mais. Em vez disso, utilizamos o **Padrão Front Controller**.



Por que isolar o diretório `public/`?

Por razões estritas de **Segurança da Informação**:

1. O servidor web aponta sua raiz estritamente para a pasta `public/`.
2. As pastas `src/`, `config/`, `vendor/` e os arquivos `.ini` e `.env` ficam **um nível acima da raiz pública**.
3. Mesmo que um atacante tente digitar diretamente a URL de um arquivo sensível (ex.: `http://site.com/config/database.ini`), o servidor web bloqueia o acesso, pois o arquivo não reside na pasta pública.

Semana 13 - Introdução ao Framework Laravel e Artisan CLI

Tema: Módulo 3 - Framework Laravel, Evolução da Arquitetura MVC Artesanal para MVC do Ecossistema Laravel, Estrutura Enxuta, Mapeamento de Diretórios. Gestão Segura de Ambientes com `.env`, Chaves Criptografadas de aplicação, Ferramenta de Linha de Comando `Artisan CLI`, Ciclo de Vida de Requisição do Framework.

O que é o Framework Laravel?

Criado em 2011 por Taylor Otwell, o **Laravel** é atualmente o framework mais popular e consolidade do ecossistema PHP. Ele é um **framework MVC Completo**, que oferece uma estrutura de desenvolvimento robusta, segura e escalável para aplicações web modernas e corporativas. Ele Adota o Princípio de **Convenção sobre Configuração** e Entrega uma série de recursos prontos para uso, incluindo:

1. **Produtividade Imediata**: Estrutura padronizada onde todo desenvolvimento segue convenções claras, reduzindo a necessidade de configuração manual.
2. **Segurança por Padrão(Security by Default)**: Proteção nativa e automática contra XSS, CSRF, SQL Injetion, manipulação indevida de cookies e ataques de força brute.
3. **ORM Eloquent**: Uma ferramenta de mapeamento objeto-relacional (ORM) que simplifica a interação com banco de dados, permitindo consultas e manipulações de dados usando sintaxe PHP fluente.
4. **Ferramenta de Linha de Comando (Artisan CLI)**: Automatiza a criação de classes, execução de migrações de banco, limpeza de cache e outras tarefas repetidas, aumento de produtividade no desenvolvimento.

5. **Comunidade Global e Ecossistema Rico**: Milhares de pacotes prontos para autenticação robusta, mensageria, fila assincronas, geração de APIs restful, testes automatizados e muito mais.

Ao longo das últimas semanas, estudamos a estrutura de projetos em PHP puro. E agora, vamos migrar para o **Laravel**, abaixo uma tabela comparativa entre o MVC artesanal e o Laravel:

Componente Arquitetural	No Seu MVC Artesanal (Semana 12)	No Framework Laravel (Semana 13+)	Vantagem Competitiva no Mercado
Ponto Único de Entrada	`public/index.php`	`public/index.php`	Mesma raiz segura: isola todo o código-fonte da web pública.
Autoload de Classes	`composer.json` (PSR-4: `"App\\": "src/"`)	`composer.json` (PSR-4: `"App\\": "app/"`)	Padrão unificado internacional da PHP-FIG.
Mecanismo de Roteamento	`App\Core\Roteador` com array interno	`routes/web.php` e `routes/console.php`	Roteamento declarativo com middlewares, parâmetros dinâmicos e grupos de rotas.
Camada de Controle	`App\Controllers\ProdutoController`	`App\Http\Controllers\AtivoController`	Integração automática com Injeção de Dependências e validação de Form Requests.
Camada de Dados (Model)	`App\Models\Produto` com PDO manual	`App\Models\Ativo` (*Eloquent ORM*)	Mapeamento Objeto-Relacional avançado com relacionamentos expressivos (`hasMany`, `belongsTo`).
Camada de Visualização	Scripts PHP nativos com `extract()`	Motor de Templates **Blade** (`.blade.php`)	Sintaxe limpa, herança de layouts, componentes reutilizáveis e escape automático contra XSS.
Isolamento de Credenciais	`config/database.ini`	Arquivo `.env` (Padrão 12-Factor App)	Paridade entre ambientes (local, homologação e

Componente Arquitetural	No Seu MVC Artesanal (Semana 12)	No Framework Laravel (Semana 13+)	Vantagem Competitiva no Mercado
			nuvem) com segurança estrita.

> [!Observação] Construir o MVC Artesanal na semana 12 foi como desmontar e criar a mão cada engrenagem, pistão e freio de um motor automotivo. Agora que você compreendeu como montar funciona, não precisa mais reinventar a roda em cada nova fase, você já assume a cabine do piloto de Fórmula 1 de última geração.

Rotas Enxutas e Parametrizadas com Laravel 11

No **Lavarel 11**, lançado com foco em performance, e simplicidade, a arquitetura foi completamente desidratada e enxuta. Permitindo que o desenvolvedor foque mais na lógica de negócio e menos em configurações complexas. Abaixo, um exemplo de rotas simples e parametrizadas:

```
use App\Http\Controllers\AtivoController;

Route::get('/', [AtivoController::class, 'index'])->name('ativos.index');
Route::get('/ativos/create', [AtivoController::class, 'create'])->name('ativos.create');
Route::post('/ativos', [AtivoController::class, 'store'])->name('ativos.store');
Route::get('/ativos/{ativo}/edit', [AtivoController::class, 'edit   '])->name('ativos.edit');
Route::put('/ativos/{ativo}', [AtivoController::class, 'update'])->name('ativos.update');
Route::delete('/ativos/{ativo}', [AtivoController::class, 'destroy'])->name
('ativos.destroy');
```

As 3 Grandes Mudanças Estruturais do Laravel 11:

1. **O Novo Núcleo `bootstrap/app.php`:** Toda a configuração de rotas (`web`, `commands`), middlewares globais e tratamento centralizado de exceções agora é feita de forma fluida (*fluent API*) em um único arquivo.
2. **Rotas Minimalistas por Padrão:** O arquivo `routes/web.php` já vem pronto para rotas de interface web. As rotas de API (`routes/api.php`) só são criadas quando o desenvolvedor decide ativá-las através do comando `php artisan install:api`.
3. **Menos Boilerplate em Controllers e Models:** Classes base limpas, sem heranças desnecessárias de dezenas de traits que não estavam sendo utilizados.

3. O Mapeamento de Diretórios do Laravel

Ao criar uma nova aplicação Laravel, a estrutura de diretórios é organizada de forma a separar claramente as responsabilidades do sistema. Usando o Princípio de Responsabbilidade Única**.

```
senai-asset-manager/
├── app/                  <- O coração da sua lógica de negócios
│   ├── Http/
│   │   └── Controllers/  <- Seus Controllers residem aqui
│   ├── Models/           <- Seus Modelos Eloquent residem aqui
│   └── Providers/        <- Provedores de serviços da aplicação
├── bootstrap/            <- Inicialização do framework
│   ├── app.php           <- Ponto central de configuração de rotas e middlewares
│   └── cache/            <- Caches de inicialização do sistema
├── config/               <- Arquivos de configuração publicáveis (banco, sessão, etc.)
├── database/             <- Engenharia de Banco de Dados
│   ├── factories/        <- Fábricas de dados falsos para testes (Faker)
```

```
│   ├── migrations/       <- Controle de versão do esquema do banco de dados
│   └── seeders/          <- Scripts para popular tabelas com dados iniciais
├── public/               <- A ÚNICA PASTA ACESSÍVEL PELO SERVIDOR WEB
│   ├── css/              <- Folhas de estilo compiladas
│   ├── js/               <- Scripts front-end
│   ├── index.php         <- O Front Controller da aplicação
│   └── robots.txt
├── resources/            <- Recursos brutos da interface com o usuário
│   ├── css/              <- Arquivos CSS pré-compilação (Tailwind)
│   ├── js/               <- Arquivos JavaScript pré-compilação
│   └── views/            <- Templates do motor de visualização Blade (.blade.php)
├── routes/               <- Mapeamento de rotas do sistema
│   ├── console.php       <- Comandos de terminal customizados baseados em Closure
│   └── web.php           <- Rotas HTTP para navegadores (com cookies e sessões)
├── storage/              <- Arquivos gerados em tempo de execução
│   ├── app/              <- Uploads e arquivos gerados pelo sistema
│   ├── framework/        <- Sessões de arquivo, views compiladas e caches
│   └── logs/             <- Logs cronológicos de erro e auditoria (laravel.log)
├── tests/                <- Testes automatizados unitários e de integração
├── .env                  <- ARQUIVO CONFIDENCIAL de variáveis de ambiente
├── .env.example          <- Modelo público de variáveis para a equipe
├── .gitignore            <- Impede o vazamento de segredos para o Git
├── artisan               <- O executável da CLI Artisan
└── composer.json         <- Manifesto de dependências e autoload PSR-4
```

> [!Cuidado] Apenas a pasta `public/` deve ser apontada pelo servidor web. Nunca aponte o servidor web para a raiz do projeto! Se fizer isso, qualquer visitante poderá navegar até `http://seusite.com/.env/` e baixar todas as senahs do banco de dados e chaves de API da empresa.

> [!Observação] `.env` ou `database.ini`: são arquivos para armazenamento de informações/dados sensíveis da aplicação. `.gitignore`: nesse arquivo são colocados os nomes dos arquivos e pastas que não devem ser versionados (não são exposoto no git e github).

4. Segurança no Ambiente de Desenvolvimento e Produção

Em Sistemas corporativos, uma aplicação passa por múltiplos ambientes ao longo do seu ciclo de desenvolvimento.

- **Ambiente local**: Na máquina de trabalho do desenvolvedor, com exibição detalhada de erros e banco de dados local.
- **Ambiente de Homologação**: Servidor de testes, onde a equipe de QA valida funcionalidades antes do lançamento.
- **Ambiente de Produção**: Servidor público, onde a aplicação é acessada pelos usuários finais. Aqui, erros detalhados não devem ser exibidos para o público. `APP_DEBUG` deve ser `false` para não expor informações sensíveis.

1. **O Padrão de Segurança `12-Factor`e o Arquivo `.env`**

Em vez de alterar senhas dentro do código PHP a cada deploy, o Laravel utiliza o padrão internacional de isolar as variáveis que mudam de ambiente em um arquivo de texto simples chamado `.env` (Environments Variables).

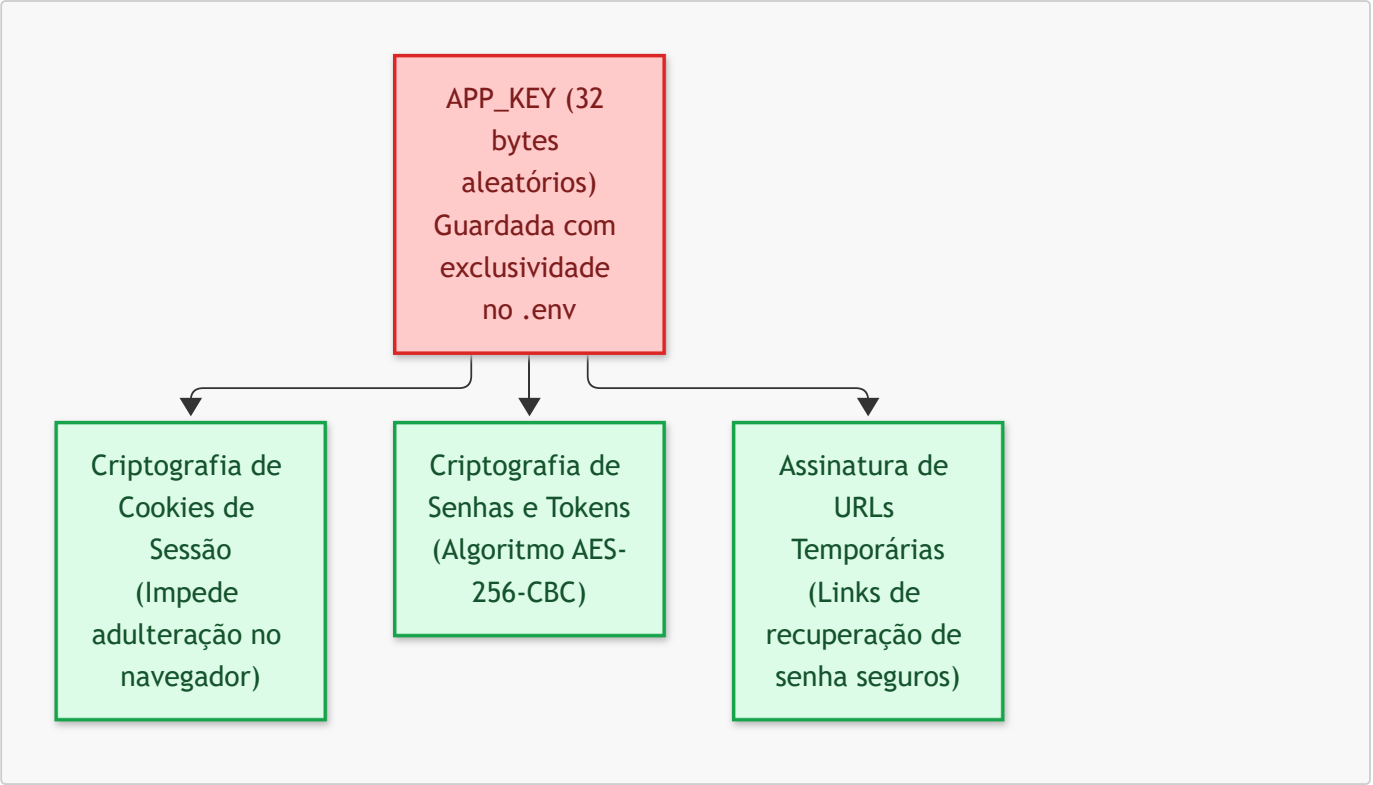
2. **A Chave de Segurança `APP_KEY`**.

A `APP_KEY` é a sequencia criptografada aleatória de 32 bytes codificada em Base64 gerada automaticamente pelo Laravel, como padrão de segurança em qualquer aplicação.

> [!Observação] Se a `APP_KEY` estiver vazia, o Laravel recusará a execução exibindo um erro na tela *No Application encryption key has been specified* Se ocorrer a troca da `APP_KEY` em um sistema em produção, todas as sessões ativas cairão e os dados cifrados com a chave antiga se tornarão inválidos e irrecuperáveis.

3. **O Papel do `.env.example`**

Como o arquivo `.env`está listado no `.gitiognore` e nunca vai para o GitHub, como um colega de equipe sabe quais variáveis ele precisa configurar ao clonar o repositório? Através do `.env.example`, ele serve como um template documentado contendo as mesma chaves do `.env`, porrém com valores fictícios ou vazios.



5. Artisan CLI: O poder da Linha de Comando

O **Artisan CLI** é a interface de linha de comando (Command-Line-Interface) nativa do Laravel. Ele é executado a partir do arquivo executável `artisan` localizado na raiz do projeto.

```
php artisan [comando] [argumentos] [opções]
```

Principais Comandos do Artisan CLI para a Rotina de Desenvolvimento

1. `php artisan serve` Inicializa o servidor HTTP local embutido de desenvolvimento, escutando por padrão em `http://127.0.0.1:8000`. É muito superior ao `php -S localhost:8000 -t public` tradicional porque já configura automaticamente o roteamento de assets estáticos e tratamento de reescrita de URL.

- Para mudar a porta: `php artisan serve --port=8080`

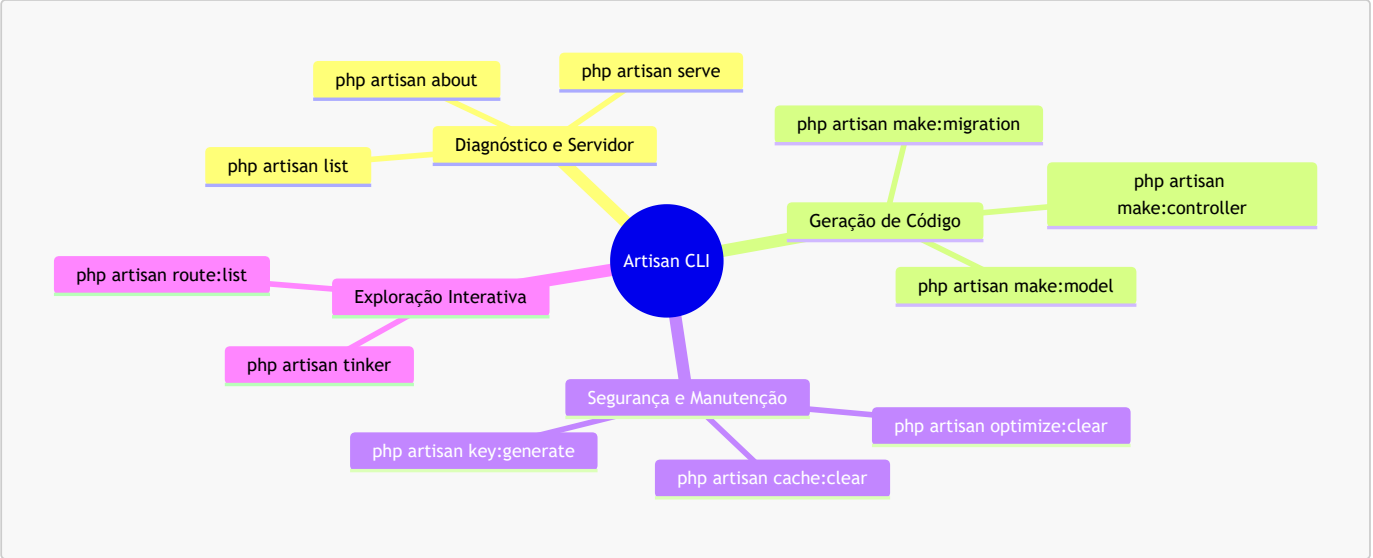
2. `php artisan key:generate` Gera uma chave criptográfica segura de 256 bits através de gerador de números pseudoaleatórios criptograficamente seguros e grava diretamente na variável `APP_KEY` do seu arquivo `.env`.

3. `php artisan about` Introduzido nas versões modernas do Laravel, este comando faz uma varredura completa da saúde do ecossistema e imprime uma tabela no terminal com:

- Versão do PHP e extensões carregadas.
- Versão do Laravel instalada.
- Status do ambiente (`local`, `production`).
- Status do modo de depuração (`Debug Mode: ENABLED/DISABLED`).
- Drivers ativos de Banco de Dados, Cache, Sessão e Filas.

4. `php artisan list` Exibe o catálogo completo de comandos disponíveis na aplicação, agrupados por categorias (`make:`, `migrate:`, `route:`, `db:`, `config:`).

5. `php artisan make:controller NomeDoController` Gera automaticamente um novo arquivo de classe Controller já configurado com o namespace correto (`App\Http\Controllers`), herança da classe base e declarações estritas, eliminando erros manuais de digitação.
6. `php artisan make:model NomeDoModel` Gera automaticamente a classe de modelo Eloquent sob a pasta `App\Models`.
7. `php artisan route:list` Gera uma tabela formatada no terminal listando todas as rotas registradas na aplicação, incluindo os verbos HTTP correspondentes (GET, POST), URIs, nomes de rota e a respectiva ação no Controller.
8. `php artisan tinker` Abre um terminal interativo (*REPL - Read-Eval-Print Loop*) baseado no PsySH. Permite testar comandos PHP, instanciar models, consultar bancos de dados e verificar configurações diretamente pela linha de comando com toda a aplicação Laravel inicializada em memória!
9. `php artisan migrate` Executa todas as migrações pendentes, criando ou alterando tabelas no banco de dados conforme definido nos arquivos de migração sob as pastas `database/migrations/`.



Semana 14 - Rotas HTTP, Controllers de Recursos e Injeção de Dependências no LARAVEL

Tema: Roteamento HTTP, Arquitetura Api Restful (GET, POST, PUT/PATCH, DELETE), Parâmetros de rotas(), Rotas Nomeadas name(), Controllers de Recursos(Parametrização do CRUD), Injeção de Dependências Via Service Container, Proteção Nativa Contra CRSF, Inspeção de rotas com `php artisan route:list`.

1. O Protocolo HTTP

Em Aplicações web legadas ou amadoras, é comum encontrar estruturas de URLs desorganizadas e perigosas que violam as boas práticas de engenharia e de segurança da informação.

Exemplo de Rota Insegura:

```
<!-- Exemplo Legado Vulnerável: Ações destrutivas executadas via GET -->
<a href="excluir_cliente.php?id=10">Excluir Cliente</a>
```

Quando uma ação destrutiva (como deletar ou alterar dados) é mapeada para o verbo **GET**, a aplicação fica exposta a desatre silencioso como:

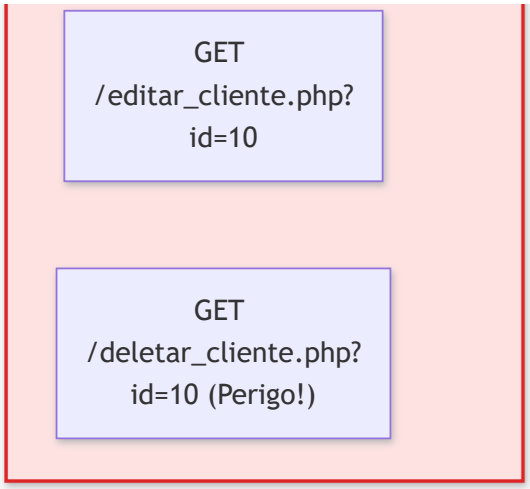
1. **Robôs de busca e Indexadores(Web Crawlers)**: Rôbos como o Googlebot ou indexadores internos casculham links `<a>` da página. Se encontrarem o link de exclusão, eles seguirão a URL e *apagarão todos os registro do seu banco de*

dados automaticamente!

2. **Pré-Carregamento do Navegador(Browser Prefetching)**: Navegadores modernos tentatm acelerar a navegação fazendo requisições em segundo plano para os links vídíveis na tela. Se o link de deletar estiver lá, a conta do cliente é excluida antes mesmo de o úsuário clicar.
3. **Histórico e Compartilhamento**: URLs GET com parâmetros destrutivos ficam salvas no histórico do navegador, em favoritos e em logs de servidores proxy.

Diferença de Construções Api com Arquitetura Restful no LARAVEL Vs. Abordagem PHP Vanila com Códigos Inseguros


Arquitetura RESTful no Laravel 11 (Padronizada)
GET /clientes/{id} (Detalhar)
GET /clientes (Listar)
POST /clientes (Criar)
PUT /clientes/{id} (Atualizar)
DELETE /clientes/{id} (Excluir Seguro)
Abordagem Legada (Insegura e Caótica)
POST /salvar_cliente.php
GET /listar_clientes.php
GET /cadastrar_cliente.php



A Filosofia de arquitetura REST (*Represent State Transfer*): Na arquitetura RESTful adotada pelo Laravel:

- A **URI** (o endereço) representa o **recurso** no plural: `/clientes`, `/ativos`. `/peças`.
- O **Verbo HTTP** indica a **ação** que você deseja executar sobre aquele recurso.
- O mesmo endereço `/cliente/10` tem significado completamentee diferentes e seguro dependendo do método HTPP enviado pelo CLiente

A Matriz de Verbos HTTP no LARAVEL

O protocolo HTTP/1.1 define verbos com semânticas bem delineadas que o Laravel implementa de forma declarativa e expressiva:

Verbo HTTP	Intenção de Engenharia	É Seguro?	É Idempotente?	Uso Típico no Laravel
GET	Recuperar dados sem causar efeitos colaterais no servidor.	**Sim**	**Sim**	Exibir telas, consultar listagens e buscar detalhes.
POST	Criar um novo recurso subordinado no servidor.	**Não**	**Não**	Submeter formulários de cadastro e criar registros.
PUT	Substituir integralmente um recurso existente no servidor.	**Não**	**Sim**	Atualizar todos os dados cadastrais de um cliente.
PATCH	Modificar parcialmente campos específicos de um recurso.	**Não**	**Não**	Alterar apenas o status de um cliente (ex.: ATIVO para INATIVO).
DELETE	Remover ou inativar permanentemente o recurso especificado.	**Não**	**Sim**	Excluir um cliente ou contrato de manutenção.

> **Conceito de Idempotência** Uma requisição é diata **idempotente** quando executada uma única vez ou dez vezes consecutiva e resultar exatamente no mesma resposta pelo servidor `DELETE /clientes/10` é idempotente (após a primeira exclusão, o registro continua excluido). `POST /clientes` **não** é idempotente (sem você clicar dez vezes, criará dez registros duplicados se não houver trava de duplicidade, nas regras de negócio).

---

2. Roteamento Declarativo e Parâmetros no Laravel

No arquvio `route/web.php`, as rotas são definidas de forma fluida através da classe fachada `Illuminate\Support\Facades\Route`.

Exemplo de Parâmetros Obrigatórios e Opcionais em Rotas GET:

```php
use Illuminate\Support\Facades\Route;

// 1. Parametro obrigatorio: {id}
Route::get('/clientes/{id}', function (int $id) {
    return "Exibindo ficha tecnica do cliente #{$id}";
});

// 2. Parametro opcional: {setor?} (requer valor padrao na variavel)
Route::get('/clientes/setor/{setor?}', function (?string $setor = 'todos') {
    return "Filtrando parceiros do setor industrial: {$setor}";
});
```

> Se alguém tentar acessar `/clientes/abc`, o roteador do Laravel rejeita a requisição imediatamente com o código de resposta **HTTP 404 (Not Found)**, sem gastar processamento de banco de dados nem invocar o Controller!

Exemplo de Validação de Restrição em Rotas GET: você pode impedir que requisições com dados inválidos cheguem ao controller utilizando métodos de restrições encadeadas

```php
// Garante que {id} seja composto estritamente por numeros decimais
Route::get('/clientes/{id}', [ClienteController::class, 'show'])
    ->whereNumber('id');

// Garante que {codigo} tenha formato alfanumerico (letras e numeros)
Route::get('/ativos/{codigo}', [AtivoController::class, 'show'])
    ->whereAlphaNumeric('codigo');

// Restricao personalizada com expressao regular (ex.: CNPJ apenas numeros, 14 digitos)
Route::get('/clientes/cnpj/{cnpj}', [ClienteController::class, 'buscarPorCnpj'])
    ->where('cnpj', '[0-9]{14}');
```

> Se alguém tentar acessar `/clientes/abc`, o roteador do Laravel rejeita a requisição imediatamente com o código de resposta **HTTP 404 (Not Found)**, sem gastar processamento de banco de dados nem invocar o Controller!

Exemplo Rotas Nomeadas (`name()`): Nunca digite URLs diretamente no HTML, como . Se for necessário alterar a rota , é necessário alterar dezena/centanas de arquivos. Com **Rotas Nomeadas**, você atribui um apelido único a rota e usa o helper gloval `route()`.

```php
// Definicao no routes/web.php:
Route::get('/clientes/{id}/editar', [ClienteController::class, 'edit'])
    ->name('clientes.edit');

// Utilizacao no template Blade ou no Controller:
// Gera automaticamente: http://localhost:8000/clientes/10/editar
$url = route('clientes.edit', ['id' => 10]);
```

4. Agrupamento de Rotas (`Route::prefix` e `Route::name`):

Organiza módulos corporativos mantendo o código limpo e sem repetições:

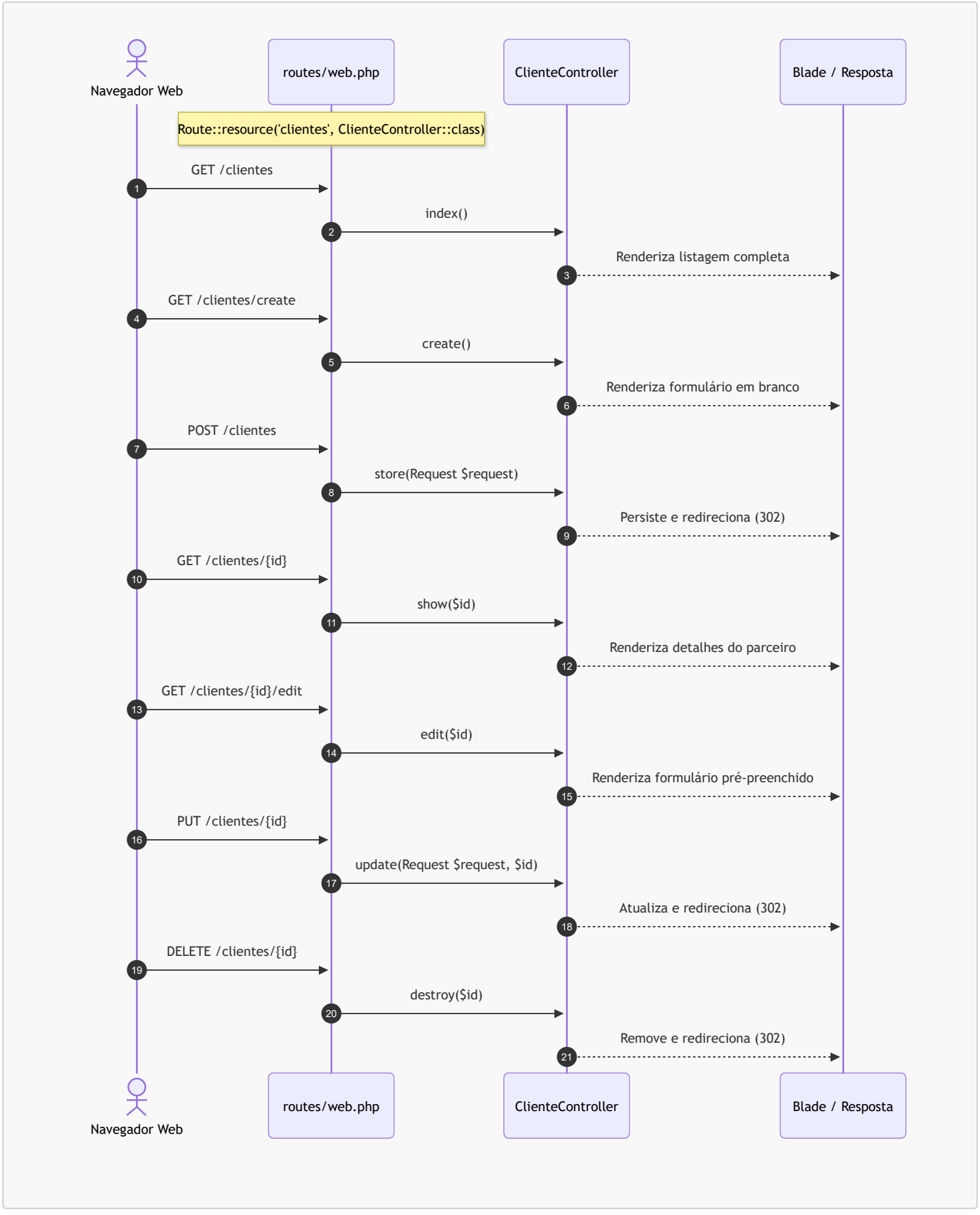
```php
Route::prefix('industrial')->name('industrial.')->group(function ()
    Route::get('/painel', [PainelController::class, 'index'])->name('painel');
    Route::get('/telemetria', [PainelController::class, 'telemetria'])->name('telemetria');
```

```
});
// Rota resultante: GET /industrial/telemetria -> route('industrial.telemetria')
```

---

3. Controllers de Recursos

Em Aplicações corporativas, praticamente todas as entidade de negócio (Clientes, Peças, Máquinas, Contratos, Movimentações) necessitam da mesmas operações fundamentais (**Listar, Exibir, Detalhar, Criar, Editar, Atualizar e Excluir**) -> CRUD



Geração via Artisan CLI

```
php artisan make:constroller ClienteController --resource
```

O comando acima cria o arquivo `app/Http/Controllers/ClienteController.php` já com as 7 assinaturas de método prontas para implementação

Mapeamento de Rotas Única //routes/web.php `Route::resource("clientes",ClienteController::class);`

Inspeção das Rotas no Terminal

```
php artisan route:list
```

```
  GET|HEAD        clientes ..................... clientes.index › ClienteController@index
  POST            clientes ..................... clientes.store › ClienteController@store
  GET|HEAD        clientes/create .............. clientes.create › ClienteController@create
  GET|HEAD        clientes/{cliente} ......... clientes.show › ClienteController@show
  PUT|PATCH       clientes/{cliente} ......... clientes.update › ClienteController@update
  DELETE          clientes/{cliente} ......... clientes.destroy › ClienteController@destroy
  GET|HEAD        clientes/{cliente}/edit .... clientes.edit › ClienteController@edit
```

Leitura de Informações das Páginas

No PHP procedural, acessa-se dados brutos através de superglobais inseguras: `$_POST["name"]` ou `$_GET["name"]`. No Laravel, essa abordagem é expressamente proibida. Em vez disso, usamos o **Service Container** do framework através da **Injeção de Dependência por Tipo**

Exemplo Prático: Basta declarar o parâmetro `Request $request`na assinatura do método do Controller. O Laravel inspeciona a assinatura via *Preflection API*, instancia o objeto com os dados da requisição e entrega pronto para uso.

```php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Http\RedirectResponse;

final class ClienteController extends Controller {
    /**
     * Salva o novo cliente injetando a requisicao HTTP com tipagem estrita.
     */
    public function store(Request $request): RedirectResponse {
        // 1. Validacao rapida de integridade
        $validados = $request->validate([
            'razao_social' => 'required|min:3',
            'cnpj'         => 'required'
        ]);

        // 2. Extracao segura de campos
        $razaoSocial = (string)$request->input('razao_social');
        $cnpj = (string)$request->input('cnpj');

        // 3. Verificacao de presenca de dados
        if ($request->has('email')) {
            $email = (string)$request->input('email');
        }

        return redirect()->route('clientes.index');
    }
}
```

4. Segurança em Formulários

Proteção Contra CRSF(Cross-Site Request Forgery) O ataque CSRF ocorre quando um site malicioso força o navegador de um usuário autenticado a enviar comando indesejados para a sua aplicação sem o consentimento dele. O Laravel neutraliza esse ataque gereando um otken criptografado único por sessão. Todo Formulário POST, PUT ou DELETE deve incluir a diretiva Blade:

```
<form action="{{ route('clientes.store') }}" method="POST">
    @csrf
    <!-- O Blade gera: <input type="hidden" name="_token" value="xY8z..."> -->
</form>
```

Simulação do método em Formulários (Method Spoofing) A especificação do HTML5 possui um limitação histórica conhecida: *formulários HTML suportam nativamente apenas o método `GET` e `POST` no atributo `method=`.

Como enviar requisições `PUT`para atualzia ou `DELETE´`para excluir dados através de formulários web padrão?

O Laravel resolve isso através de técnica de **Method Spoofing**: que define como `method="POST"` e injeta o campo oculto `_metrhod`através da diretiva Blade `@method("verbo")`:

Exemplos Práticos:

- Formulário de Atualização (PUT):

```
<form action="{{ route('clientes.update', $cliente['id']) }}" method="POST">
    @csrf
    @method('PUT')
    <!-- O Blade gera: <input type="hidden" name="_method" value="PUT"> -->
    <button type="submit">Salvar Alterações</button>
</form>
```

- Botão de Exclusão Destrutiva (DELETE):

```
<form action="{{ route('clientes.destroy', $cliente['id']) }}" method="POST" onsubmit="return
confirm('Confirmar exclusão?');">
    @csrf
    @method('DELETE')
    <button type="submit" style="color:red;">Excluir Parceiro</button>
</form>
```